บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

การซื้อขายสินค้าผ่านการสนทนาทางแชทได้รับความนิยมอย่างแพร่หลายในประเทศไทย ผู้บริโภคติดต่อสอบถามรายละเอียด ขอชมรูปภาพสินค้า และตกลงซื้อขายกับร้านค้าผ่านสื่อสังคมออนไลน์ ได้แก่ LINE Official Account, Facebook Messenger และ Instagram ร้านค้าออนไลน์จึงจำเป็นต้องเปิดให้บริการพร้อมกันหลายช่องทางเพื่อรองรับกลุ่มลูกค้าได้อย่างทั่วถึง

การสื่อสารและการติดตามงานผ่านสมาร์ตโฟนมีความจำเป็นอย่างยิ่งต่อผู้ประกอบการและพนักงานที่ต้องตอบสนองลูกค้าได้อย่างรวดเร็วตลอดเวลา แต่เนื่องจากแต่ละแพลตฟอร์มมีแอปพลิเคชันแยกจากกัน เจ้าหน้าที่จึงต้องสลับเปิดใช้งานหลายแอปพลิเคชันบนสมาร์ตโฟน ส่งผลให้การแจ้งเตือนตกหล่นและตอบสนองลูกค้าล่าช้า เมื่อมีเจ้าหน้าที่หลายคนช่วยกันดูแลก็เกิดปัญหาตอบข้อความซ้ำซ้อนเนื่องจากไม่มีระบบแบ่งมอบหมายงาน อีกทั้งการเปิดคำสั่งซื้อยังกระทำโดยจดบันทึกแยกส่วน จึงเสี่ยงต่อความผิดพลาดและไม่สามารถติดตามสถานะคำสั่งซื้อที่ผูกกับบทสนทนาได้อย่างเป็นระบบ

จากปัญหาดังกล่าว ผู้จัดทำจึงพัฒนาแอปพลิเคชัน Chatshop ซึ่งเป็นแอปพลิเคชันบนสมาร์ตโฟนที่พัฒนาด้วย Flutter สำหรับเชื่อมต่อกับระบบบริการหลังบ้าน เพื่อรวมศูนย์การรับส่งข้อความจากทุกช่องทางไว้ในกล่องข้อความเดียวแบบเรียลไทม์ พร้อมระบบแจ้งเตือนบนสมาร์ตโฟนแบบ Push Notification เมื่อมีข้อความใหม่หรือแชทค้างตอบ การจัดการสถานะบทสนทนา การกำหนดบทบาทและสิทธิ์ของทีมงาน การจัดการข้อมูลสินค้า ลูกค้า และการเปิดคำสั่งซื้อได้โดยตรงจากหน้าต่างสนทนา รวมถึงการควบคุมโหมดการทำงานของแชทบอทเพื่อช่วยตอบคำถามลูกค้าได้อย่างรวดเร็ว

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาแอปพลิเคชันบนสมาร์ตโฟนด้วย Flutter ที่สามารถรวบรวมและตอบกลับข้อความจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ได้ในหน้าจอเดียวแบบเรียลไทม์

1.2.2 เพื่อพัฒนาระบบรับส่งข้อมูลแบบสองทางผ่าน WebSocket และระบบแจ้งเตือน Push Notification บนสมาร์ตโฟนผ่าน Firebase Cloud Messaging เพื่อให้เจ้าหน้าที่รับทราบข้อความใหม่และการแจ้งเตือนแชทที่ค้างตอบได้อย่างทันท่วงที

1.2.3 เพื่อพัฒนาระบบจัดการบทสนทนา การมอบหมายผู้รับผิดชอบ การคัดกรองและค้นหาบทสนทนา และการควบคุมโหมดการทำงานของแชทบอทผ่านแอปพลิเคชัน

1.2.4 เพื่อพัฒนาระบบจัดการข้อมูลสินค้า ลูกค้า และการเปิดคำสั่งซื้อที่เชื่อมโยงกับบทสนทนาบนแอปพลิเคชัน

1.2.5 เพื่อเชื่อมต่อแอปพลิเคชันบนสมาร์ตโฟนเข้ากับระบบบริการหลังบ้านผ่านส่วนต่อประสานโปรแกรมประยุกต์และระบบยืนยันตัวตนที่มีความปลอดภัย

1.3 ขอบเขตของโครงงาน

1.3.1 แอปพลิเคชันพัฒนาด้วย Flutter และภาษา Dart รองรับการทำงานบนระบบปฏิบัติการ Android และเชื่อมต่อกับระบบบริการหลังบ้านผ่านส่วนต่อประสานโปรแกรมประยุกต์

1.3.2 แอปพลิเคชันยืนยันตัวตนผู้ใช้งานด้วยระบบอีเมลร่วมกับรหัสผ่าน และรองรับการเข้าสู่ระบบด้วยบัญชี Facebook และ Google พร้อมจัดเก็บโทเคนความปลอดภัยบนอุปกรณ์

1.3.3 เจ้าหน้าที่สามารถดูรายการบทสนทนารวมทุกช่องทาง คัดกรองตามช่องทางหรือข้อความที่ยังไม่อ่าน และค้นหาบทสนทนาจากชื่อ ข้อความ หรือแท็กได้จากกล่องข้อความเดียว

1.3.4 เจ้าหน้าที่สามารถตอบกลับข้อความ ส่งรูปภาพ วิดีโอ และไฟล์เอกสาร รวมถึงการตอบกลับข้อความสำเร็จรูปและการอ้างอิงข้อความเดิมได้

1.3.5 ระบบแสดงข้อความเข้าใหม่แบบเรียลไทม์ผ่าน WebSocket และส่งการแจ้งเตือน Push Notification มายังสมาร์ตโฟนเมื่อมีข้อความใหม่หรือมีแชทที่ค้างตอบเกินเกณฑ์ที่กำหนด

1.3.6 เจ้าหน้าที่สามารถล็อกบทสนทนาเพื่อป้องกันการตอบซ้ำซ้อน และมอบหมายบทสนทนาให้แก่สมาชิกในทีมงานตามสิทธิ์ที่ได้รับ

1.3.7 ผู้ใช้งานสามารถเลือกปรับเปลี่ยนโหมดการทำงานของแชทบอท และเรียกดูผลการวิเคราะห์เจตนาหรือข้อมูลสรุปของบทสนทนาได้

1.3.8 เจ้าหน้าที่สามารถดูและแก้ไขข้อมูลลูกค้า จัดการแคตตาล็อกสินค้า และเปิดคำสั่งซื้อสินค้าจากหน้าต่างสนทนาได้โดยตรง

1.3.9 ผู้ใช้งานสามารถตรวจสอบรายงานภาพรวมการให้บริการและสัดส่วนปริมาณข้อความของแต่ละช่องทางได้

1.3.10 ระบบบริการหลังบ้านที่พัฒนาด้วย Laravel ทำหน้าที่ประมวลผลคำขอ จัดการฐานข้อมูล รับส่งข้อมูลกับแพลตฟอร์มภายนอก และเปิดส่วนต่อประสานโปรแกรมประยุกต์ให้แอปพลิเคชันเรียกใช้งาน

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1.4.1 เจ้าหน้าที่สามารถติดตามและตอบข้อความลูกค้าจากทุกช่องทางผ่านสมาร์ตโฟนได้สะดวกรวดเร็วในหน้าจอเดียว

1.4.2 ลดความผิดพลาดและลดความล่าช้าในการตอบสนองลูกค้าด้วยระบบแจ้งเตือน Push Notification และการสื่อสารแบบเรียลไทม์

1.4.3 เพิ่มประสิทธิภาพในการทำงานร่วมกันของทีมงานผ่านการแบ่งมอบหมายหน้าที่และการควบคุมสิทธิ์การเข้าถึงข้อมูล

1.4.4 อำนวยความสะดวกในการเปิดคำสั่งซื้อและบันทึกข้อมูลลูกค้าได้ทันทีระหว่างการสนทนา

1.4.5 ผู้ประกอบการสามารถติดตามสถานะการสนทนาและประสิทธิภาพการดำเนินงานของร้านค้าได้ทุกที่ทุกเวลา

1.5 นิยามศัพท์เฉพาะ

1.5.1 Chatshop หมายถึง แอปพลิเคชันรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์ที่พัฒนาขึ้นในโครงงานนี้ ซึ่งทำงานบนสมาร์ตโฟนและเชื่อมโยงกับระบบบริการหลังบ้าน

1.5.2 Flutter หมายถึง ชุดพัฒนาซอฟต์แวร์ส่วนต่อประสานผู้ใช้แบบข้ามแพลตฟอร์มที่พัฒนาโดย Google สำหรับสร้างแอปพลิเคชันด้วยภาษา Dart

1.5.3 Riverpod หมายถึง เครื่องมือสำหรับบริหารจัดการสถานะของแอปพลิเคชันในระบบนิเวศของ Flutter ที่มีโครงสร้างแบบไม่ขึ้นต่อบริบทของหน้าจอและรองรับการทำงานแบบไม่ประสานเวลา

1.5.4 Push Notification หมายถึง การส่งข้อความแจ้งเตือนจากเครื่องแม่ข่ายผ่าน Firebase Cloud Messaging มายังสมาร์ตโฟนของผู้ใช้งานโดยตรง แม้ไม่ได้เปิดใช้งานแอปพลิเคชันอยู่ก็ตาม

1.5.5 WebSocket หมายถึง โพรโทคอลการสื่อสารข้อมูลแบบสองทางผ่านการเชื่อมต่อแบบเปิดค้างไว้ระหว่างเครื่องแม่ข่ายกับแอปพลิเคชัน เพื่อการรับส่งข้อมูลแบบเรียลไทม์

บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การพัฒนาแอปพลิเคชัน Chatshop ประกอบด้วยแอปพลิเคชันบนสมาร์ตโฟนสำหรับเจ้าหน้าที่ร้านค้า และระบบบริการหลังบ้านสำหรับรับส่งข้อความกับแพลตฟอร์มภายนอก จัดเก็บข้อมูล และประมวลผลแชทบอท ผู้จัดทำจึงศึกษาแนวคิดและเทคโนโลยีที่เกี่ยวข้องทั้งสองส่วน เพื่อนำมาออกแบบและพัฒนาระบบให้ทำงานได้ตามวัตถุประสงค์ โดยมีหัวข้อดังนี้

2.1 Flutter

2.2 ภาษา Dart

2.3 Riverpod

2.4 go_router

2.5 Laravel

2.6 REST API

2.7 OAuth 2.0

2.8 LINE Messaging API

2.9 Meta Graph API

2.10 Webhook

2.11 Queue และ Task Scheduling

2.12 WebSocket และ Laravel Reverb

2.13 Firebase Cloud Messaging (FCM)

2.14 MySQL

2.15 MongoDB

2.16 สถาปัตยกรรม Multi-Tenant

2.17 การควบคุมสิทธิ์ตามบทบาท (RBAC)

2.18 แชทบอท

2.19 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

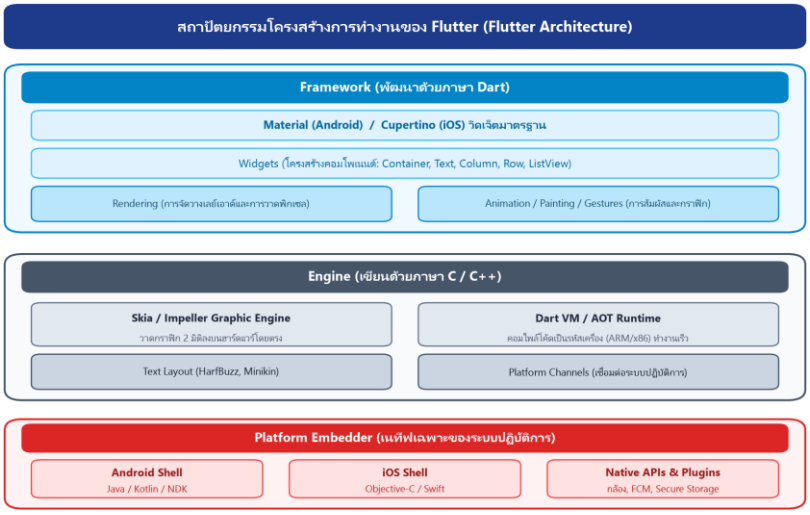
2.20 RAG (Retrieval-Augmented Generation)

2.21 Embedding และความคล้ายโคไซน์

2.22 งานวิจัยที่เกี่ยวข้อง

2.1 Flutter

Flutter คือชุดพัฒนาซอฟต์แวร์แบบโอเพนซอร์สของ Google สำหรับสร้างแอปพลิเคชันหลายแพลตฟอร์มจากโค้ดชุดเดียว [1] ส่วนติดต่อผู้ใช้ประกอบขึ้นจากวิดเจ็ตที่ซ้อนกันเป็นลำดับชั้น และวาดหน้าจอด้วยเอนจินกราฟิกของ Flutter เอง จึงแสดงผลเหมือนกันทุกอุปกรณ์ ดังรูปที่ 2.1 ระหว่างพัฒนารองรับ Hot Reload ที่แสดงผลการแก้ไขโค้ดได้ทันทีโดยไม่ต้องเริ่มแอปพลิเคชันใหม่



รูปที่ 2.1 โครงสร้างการทำงานของ Flutter

2.1.1 ข้อดี

2.1.1.1 เขียนโค้ดชุดเดียวแล้วนำไปใช้ได้ทั้ง Android และ iOS

2.1.1.2 มีวิดเจ็ตตามแนวทาง Material Design ให้ใช้งานครบถ้วน

2.1.1.3 Hot Reload ช่วยลดเวลาในการพัฒนาและปรับแต่งหน้าจอ

2.1.2 ข้อเสีย

2.1.2.1 ไฟล์ติดตั้งมีขนาดใหญ่กว่าแอปพลิเคชันแบบเนทีฟ

2.1.2.2 การใช้ความสามารถเฉพาะของอุปกรณ์ต้องพึ่งแพ็กเกจเสริม

2.1.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Flutter 3 พัฒนาแอปพลิเคชันสำหรับสมาร์ตโฟนบนระบบปฏิบัติการ Android แบ่งโค้ดเป็น 13 โมดูลตามหน้าที่ เช่น การเข้าสู่ระบบ กล่องข้อความ หน้าต่างสนทนา ลูกค้า สินค้า คำสั่งซื้อ การแจ้งเตือน รายงาน และการตั้งค่า

2.2 ภาษา Dart

Dart คือภาษาโปรแกรมเชิงวัตถุของ Google ที่ใช้พัฒนาแอปพลิเคชันด้วย Flutter [2] มีระบบชนิดข้อมูลแบบ Null Safety ที่ตรวจสอบค่าว่างตั้งแต่ขั้นตอนคอมไพล์ และรองรับการเขียนโปรแกรมแบบไม่ประสานเวลาด้วย Future Stream และคำสั่ง async กับ await ทำให้รอข้อมูลจากเครือข่ายได้โดยหน้าจอไม่ค้าง

2.2.1 ข้อดี

2.2.1.1 Null Safety ช่วยลดข้อผิดพลาดจากค่าว่างขณะแอปพลิเคชันทำงาน

2.2.1.2 คอมไพล์เป็นรหัสเครื่องสำหรับการติดตั้งจริง จึงทำงานได้รวดเร็ว

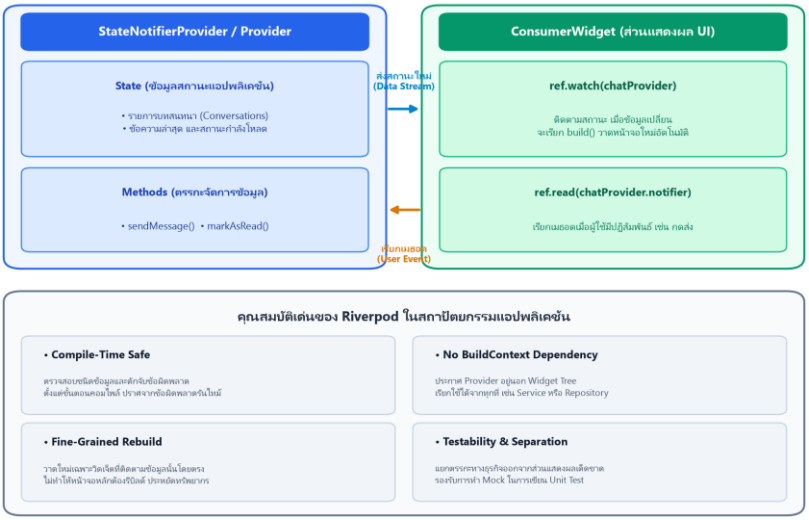
2.2.2 ข้อเสีย

2.2.2.1 มีผู้ใช้งานน้อยกว่าภาษาหลักอื่น เช่น JavaScript และ Kotlin

2.2.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Dart เขียนโค้ดทั้งหมดของแอปพลิเคชัน โดยใช้ Future สำหรับเรียก REST API ของระบบหลังบ้าน และใช้ Stream สำหรับรับเหตุการณ์แบบเรียลไทม์และการแจ้งเตือน

2.3 Riverpod

Riverpod คือไลบรารีบริหารจัดการสถานะของแอปพลิเคชันสำหรับ Flutter [3] ข้อมูลแต่ละชุดประกาศเป็น Provider ที่ไม่ขึ้นกับตำแหน่งของวิดเจ็ต วิดเจ็ตที่ติดตามข้อมูลด้วยคำสั่ง watch จะสร้างหน้าจอใหม่เมื่อข้อมูลเปลี่ยน และ Provider หนึ่งสามารถติดตาม Provider อื่นได้ เมื่อข้อมูลต้นทางเปลี่ยน ข้อมูลที่คำนวณต่อจากกันจะถูกคำนวณใหม่ตามลำดับโดยอัตโนมัติ ดังรูปที่ 2.2



รูปที่ 2.2 ความสัมพันธ์ระหว่าง Provider และวิดเจ็ตใน Riverpod

2.3.1 ข้อดี

2.3.1.1 แยกตรรกะของข้อมูลออกจากหน้าจอ ทำให้ทดสอบได้ง่าย

2.3.1.2 ตรวจสอบชนิดข้อมูลได้ตั้งแต่ขั้นตอนคอมไพล์

2.3.1.3 จัดการสถานะกำลังโหลดและข้อผิดพลาดให้โดยอัตโนมัติ

2.3.2 ข้อเสีย

2.3.2.1 ผู้พัฒนาต้องเรียนรู้แนวคิดของ Provider หลายชนิดก่อนใช้งาน

2.3.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Riverpod เก็บสถานะการเข้าสู่ระบบ ตัวเรียก REST API ข้อมูลบทสนทนา บริการเรียลไทม์ และบริการแจ้งเตือน เมื่อผู้ใช้เข้าสู่ระบบหรือออกจากระบบ Provider ที่ติดตามสถานะนี้จะเชื่อมต่อหรือยกเลิกการเชื่อมต่อเรียลไทม์และการลงทะเบียนอุปกรณ์ให้เองทันที

2.4 go_router

go_router คือแพ็กเกจกำหนดเส้นทางหน้าจอของ Flutter [4] ที่ระบุแต่ละหน้าจอด้วยเส้นทางในรูปแบบ URL เช่น /inbox/thread/:id รองรับการเปลี่ยนเส้นทางตามเงื่อนไข และการซ้อนเส้นทางไว้ภายใต้โครงหน้าจอเดียวกันด้วย ShellRoute

2.4.1 ข้อดี

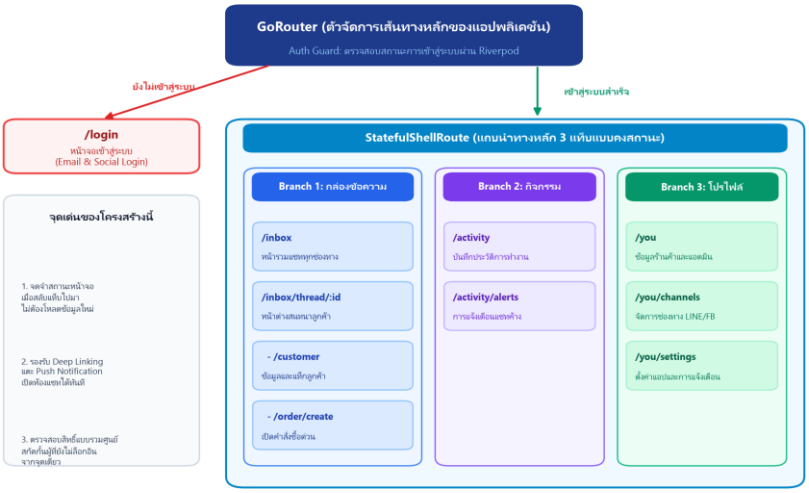
2.4.1.1 เปิดหน้าจอที่ต้องการได้โดยตรงจากเส้นทาง เช่น เมื่อแตะการแจ้งเตือน

2.4.1.2 ตรวจสอบสิทธิ์ก่อนเข้าหน้าจอได้จากจุดเดียว

2.4.2 ข้อเสีย

2.4.2.1 เส้นทางที่ซ้อนกันหลายชั้นทำให้โครงสร้างโค้ดซับซ้อนขึ้น

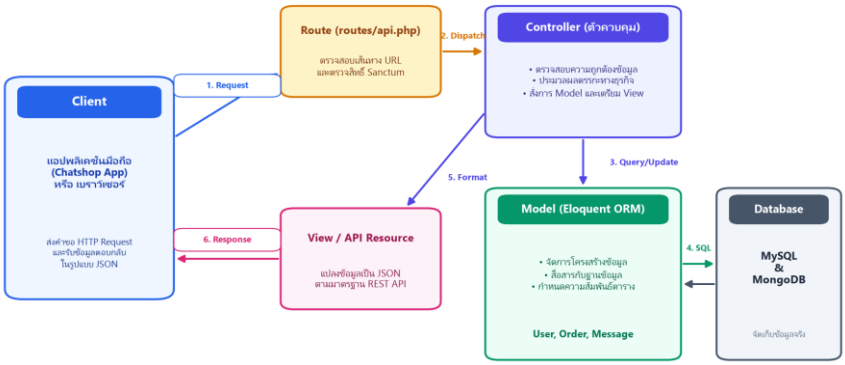
2.4.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ StatefulShellRoute สร้างแถบนำทางด้านล่าง 3 แท็บ ได้แก่ กล่องข้อความ การแจ้งเตือน และข้อมูลผู้ใช้ โดยแต่ละแท็บจำหน้าจอล่าสุดไว้เมื่อสลับแท็บ และกำหนดให้ผู้ใช้ที่ยังไม่เข้าสู่ระบบถูกส่งไปยังหน้าเข้าสู่ระบบเสมอ ดังรูปที่ 2.3



รูปที่ 2.3 โครงสร้างเส้นทางหน้าจอของแอปพลิเคชันด้วย go_router

2.5 Laravel

Laravel คือเฟรมเวิร์กภาษา PHP ที่ใช้รูปแบบ MVC (Model-View-Controller) [5] มีเครื่องมือพื้นฐานสำหรับพัฒนาระบบฝั่งเครื่องแม่ข่าย ได้แก่ การกำหนดเส้นทาง Eloquent ORM สำหรับติดต่อฐานข้อมูล ระบบยืนยันตัวตน และระบบคิวงานสำหรับประมวลผลเบื้องหลัง การทำงานตามรูปแบบ MVC แสดงดังรูปที่ 2.4



รูปที่ 2.4 การทำงานตามรูปแบบ MVC ของ Laravel

2.5.1 ข้อดี

2.5.1.1 มีเครื่องมือพื้นฐานครบ ลดเวลาในการพัฒนา

2.5.1.2 โครงสร้างโค้ดเป็นระเบียบตามรูปแบบ MVC

2.5.1.3 มีเอกสารและชุมชนผู้ใช้งานจำนวนมาก

2.5.2 ข้อเสีย

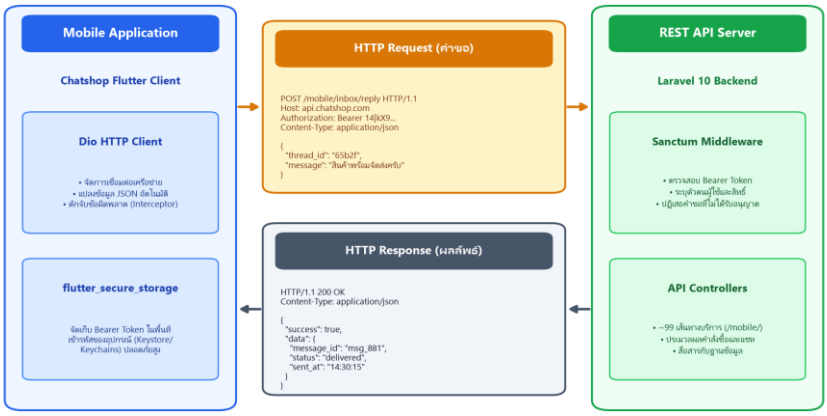
2.5.2.1 ใช้ทรัพยากรเครื่องแม่ข่ายมากกว่าการเขียน PHP แบบไม่ใช้เฟรมเวิร์ก

2.5.2.2 ผู้พัฒนาต้องเรียนรู้ข้อกำหนดของเฟรมเวิร์กก่อนใช้งาน

2.5.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Laravel 10 พัฒนาระบบบริการหลังบ้านทั้งหมด ได้แก่ การรับ Webhook การส่งข้อความออกไปยังแพลตฟอร์ม คิวงาน แชทบอท การส่งการแจ้งเตือน และ REST API ที่แอปพลิเคชันเรียกใช้

2.6 REST API

REST API คือรูปแบบการออกแบบส่วนต่อประสานที่ใช้โพรโทคอล HTTP ในการสื่อสารระหว่างระบบ [6] โดยระบุทรัพยากรด้วย URL และใช้เมธอด GET POST PUT PATCH และ DELETE แทนการอ่าน สร้าง แก้ไข และลบข้อมูล ข้อมูลที่รับส่งนิยมใช้รูปแบบ JSON และเครื่องแม่ข่ายไม่เก็บสถานะของผู้เรียก คำขอแต่ละครั้งจึงต้องแนบโทเคนยืนยันตัวตนมาด้วย ดังรูปที่ 2.5



รูปที่ 2.5 การรับส่งข้อมูลผ่าน REST API

2.6.1 ข้อดี

2.6.1.1 ใช้มาตรฐาน HTTP ที่ทุกแพลตฟอร์มรองรับ

2.6.1.2 แยกการพัฒนาแอปพลิเคชันออกจากระบบหลังบ้านได้ชัดเจน

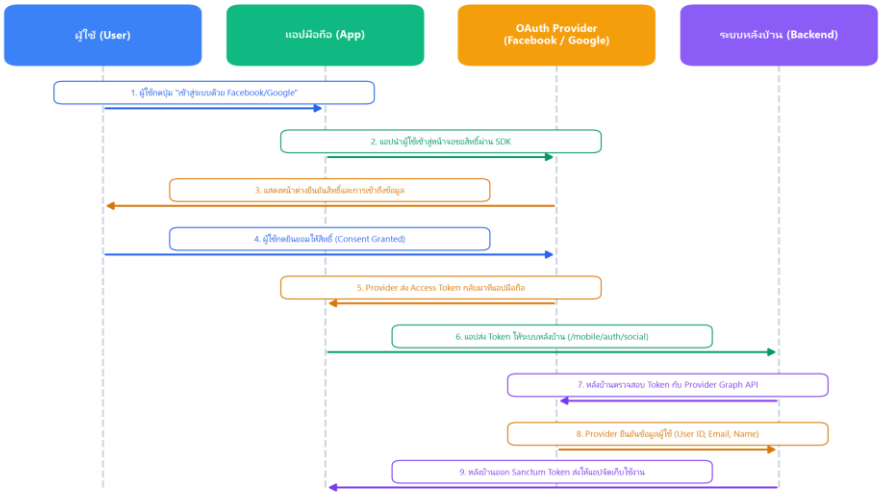
2.6.2 ข้อเสีย

2.6.2.1 หน้าจอที่ใช้ข้อมูลหลายชุดต้องเรียกหลายครั้ง

2.6.3 การนำไปใช้ในโครงการ ระบบหลังบ้านเปิด REST API ประมาณ 99 เส้นทาง ภายใต้ mobile/ และออกโทเคนด้วย Laravel Sanctum เมื่อเข้าสู่ระบบสำเร็จ แอปพลิเคชันใช้ flutter_secure_storage เก็บโทเคนไว้ในพื้นที่จัดเก็บที่เข้ารหัสของอุปกรณ์ และแนบโทเคนไปกับทุกคำขอในส่วนหัว Authorization หากระบบหลังบ้านตอบว่าโทเคนไม่ถูกต้อง แอปพลิเคชันจะลบโทเคนและกลับไปหน้าเข้าสู่ระบบ

2.7 OAuth 2.0

OAuth 2.0 คือมาตรฐานการอนุญาตให้แอปพลิเคชันเข้าถึงข้อมูลของผู้ใช้บนบริการอื่น โดยผู้ใช้ไม่ต้องให้รหัสผ่านแก่แอปพลิเคชัน [7] ผู้ใช้เข้าสู่ระบบและยินยอมที่ผู้ให้บริการ แล้วแอปพลิเคชันจะได้รับ Access Token สำหรับเรียกใช้ API แทนผู้ใช้ ดังรูปที่ 2.6

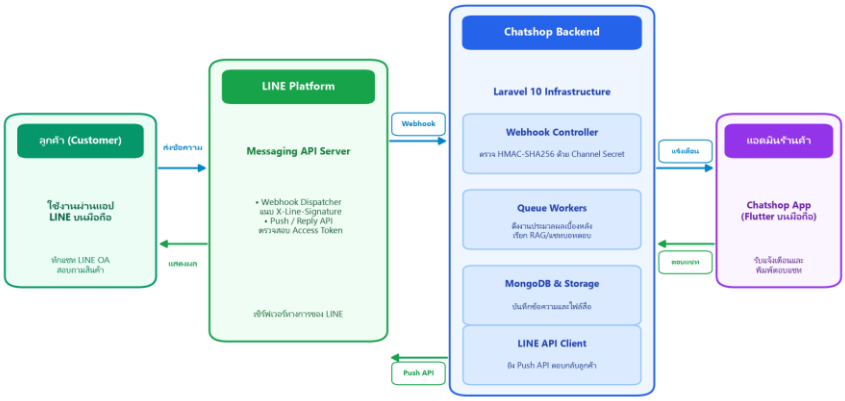


รูปที่ 2.6 ขั้นตอนการขอสิทธิ์ด้วย OAuth 2.0

2.7.1 การนำไปใช้ในโครงการ แอปพลิเคชันเรียกหน้าเข้าสู่ระบบของ Facebook และ Google บนสมาร์ตโฟนโดยตรง แล้วส่งโทเคนที่ได้ไปยังระบบหลังบ้าน ระบบหลังบ้านนำโทเคนไปตรวจสอบกับผู้ให้บริการ จากนั้นสร้างหรือค้นหาบัญชีผู้ใช้และออกโทเคนของระบบกลับมาให้แอปพลิเคชันใช้งาน นอกจากนี้ระบบหลังบ้านยังใช้ OAuth ของ Facebook ขอสิทธิ์จัดการข้อความของเพจร้านค้า

2.8 LINE Messaging API

LINE Messaging API คือบริการของ LINE สำหรับเชื่อมต่อระบบภายนอกกับบัญชี LINE Official Account [8] ระบบรับข้อความจากผู้ใช้ผ่าน Webhook และส่งข้อความกลับผ่าน API โดยยืนยันตัวตนด้วย Channel Access Token และตรวจสอบ Webhook ด้วย Channel Secret ดังรูปที่ 2.7



รูปที่ 2.7 การรับส่งข้อความผ่าน LINE Messaging API

2.8.1 ข้อดี

2.8.1.1 เข้าถึงผู้ใช้ LINE ซึ่งเป็นแอปพลิเคชันแชทที่คนไทยใช้มากที่สุด

2.8.1.2 ส่งข้อความได้หลายรูปแบบ เช่น ข้อความ รูปภาพ และสติกเกอร์

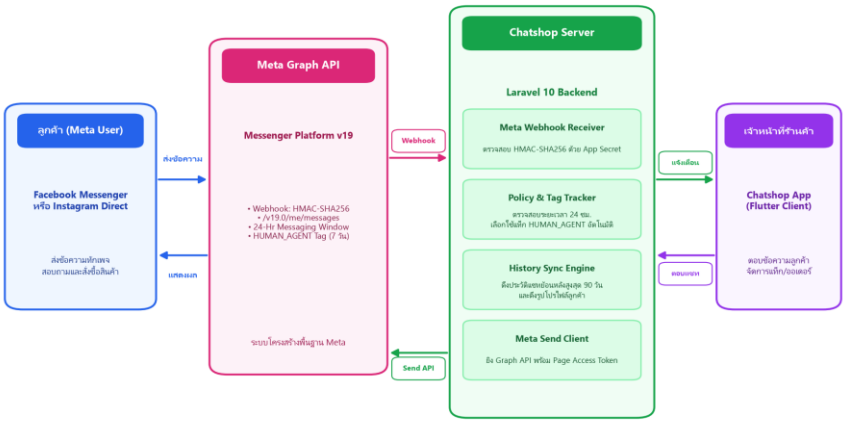
2.8.2 ข้อเสีย

2.8.2.1 ไฟล์สื่อจากผู้ใช้ไม่มี URL สาธารณะ ต้องดาวน์โหลดผ่าน API

2.8.3 การนำไปใช้ในโครงการ ระบบหลังบ้านรับข้อความจาก LINE ผ่าน Webhook ของแต่ละช่องทาง และส่งข้อความกลับด้วย Push API โดยแสดงสถานะกำลังโหลดให้ลูกค้าเห็นระหว่างที่แชทบอทสร้างคำตอบ

2.9 Meta Graph API

Meta Graph API คือ API ที่บริษัท Meta เปิดให้นักพัฒนาใช้เข้าถึงข้อมูลและบริการบนแพลตฟอร์ม Facebook และ Instagram [9] โครงงานนี้ใช้ส่วน Messenger Platform สำหรับรับส่งข้อความกับผู้ใช้ผ่านเพจ Facebook และบัญชี Instagram โดยใช้ Page Access Token ในการยืนยันตัวตน ดังรูปที่ 2.8



รูปที่ 2.8 การรับส่งข้อความผ่าน Messenger Platform

2.9.1 ข้อดี

2.9.1.1 ใช้ API ชุดเดียวส่งข้อความได้ทั้ง Messenger และ Instagram

2.9.1.2 ดึงประวัติบทสนทนาเดิมของเพจมาเก็บในระบบได้

2.9.2 ข้อเสีย

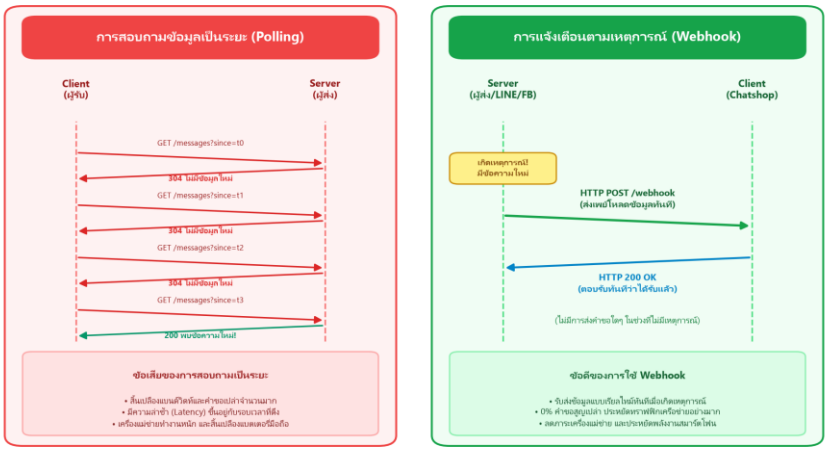
2.9.2.1 ข้อความอัตโนมัติส่งได้ภายใน 24 ชั่วโมงหลังข้อความล่าสุดเท่านั้น

2.9.2.2 โทเคนของเพจหมดอายุหรือถูกยกเลิกได้ ระบบต้องตรวจจับข้อผิดพลาดนี้ ปิดการเชื่อมต่อช่องทางนั้น และแจ้งให้ร้านค้าเชื่อมต่อใหม่

2.9.3 การนำไปใช้ในโครงการ ระบบหลังบ้านใช้ Graph API เวอร์ชัน 19.0 ส่งข้อความตอบกลับ ดึงรูปโปรไฟล์ลูกค้า และนำเข้าประวัติแชทย้อนหลังได้ไม่เกิน 90 วัน

2.10 Webhook

Webhook คือกลไกที่ระบบต้นทางส่งคำขอ HTTP ไปยัง URL ที่ระบบปลายทางได้ลงทะเบียนไว้ล่วงหน้าทันทีเมื่อเกิดเหตุการณ์ [10] เช่น มีข้อความใหม่ ระบบปลายทางจึงรับข้อมูลได้ทันทีโดยไม่ต้องสอบถามซ้ำเป็นระยะ ดังรูปที่ 2.9 ผู้ให้บริการมักแนบลายเซ็นดิจิทัลที่สร้างจาก HMAC มาด้วย เพื่อให้ระบบปลายทางตรวจสอบได้ว่าข้อมูลมาจากผู้ให้บริการจริงและไม่ถูกแก้ไขระหว่างทาง



รูปที่ 2.9 การทำงานของ Webhook เทียบกับการสอบถามข้อมูลเป็นระยะ

2.10.1 ข้อดี

2.10.1.1 รับข้อความใหม่ได้ทันที ไม่ต้องสอบถามข้อมูลซ้ำเป็นระยะ

2.10.1.2 ลดภาระของเครื่องแม่ข่ายทั้งฝั่งผู้ให้บริการและฝั่งระบบ

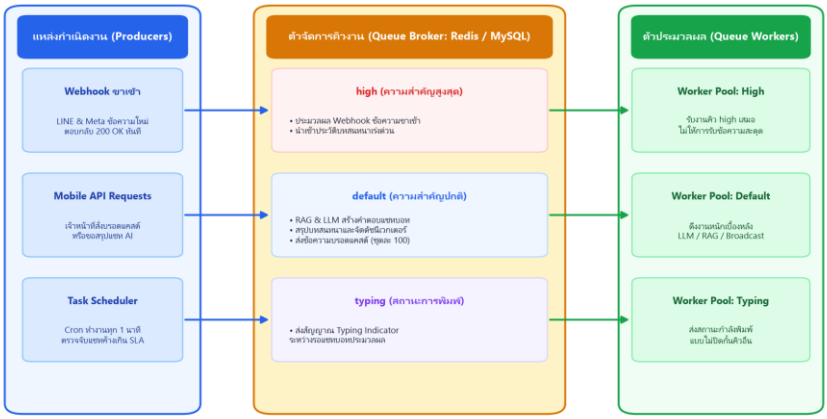
2.10.2 ข้อเสีย

2.10.2.1 ระบบต้องตอบกลับให้เร็ว หากตอบช้าผู้ให้บริการจะส่งซ้ำ

2.10.3 การนำไปใช้ในโครงการ ระบบหลังบ้านตรวจสอบลายเซ็น X-Line-Signature ของ LINE และ X-Hub-Signature-256 ของ Meta ด้วย HMAC-SHA256 จากนั้นส่งข้อมูลเข้าคิวงานทันที เพื่อให้ตอบกลับผู้ให้บริการได้รวดเร็ว แล้วจึงบันทึกข้อความและเรียกแชทบอทในเบื้องหลัง

2.11 Queue และ Task Scheduling

Queue คือการย้ายงานที่ใช้เวลานานออกไปทำงานเบื้องหลัง ทำให้ระบบตอบกลับคำขอได้ทันที ดังรูปที่ 2.10 งานแต่ละประเภทแยกไว้คนละคิวได้เพื่อกำหนดลำดับความสำคัญ ส่วน Task Scheduling คือการกำหนดให้คำสั่งทำงานอัตโนมัติตามรอบเวลา [11]



รูปที่ 2.10 การทำงานของคิวงาน

2.11.1 การนำไปใช้ในโครงการ

2.11.1.1 คิว high ใช้ประมวลผล Webhook ขาเข้าและการนำเข้าประวัติแชท

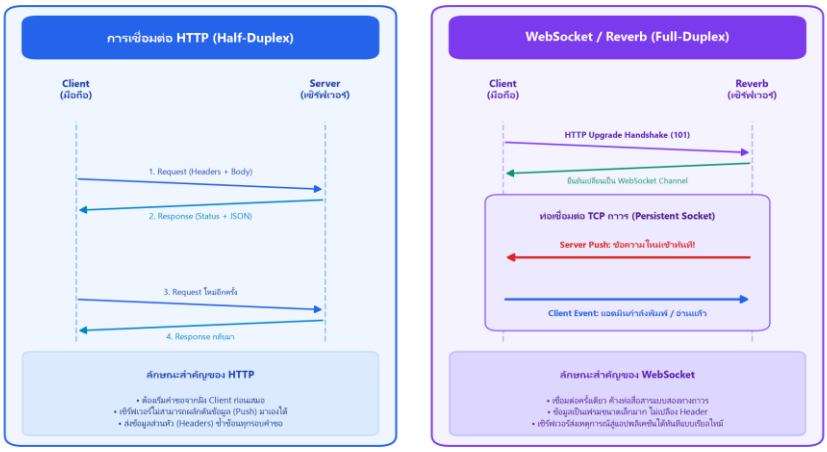
2.11.1.2 คิว default ใช้สร้างคำตอบของแชทบอท สรุปบทสนทนา จัดทำดัชนีเอกสารความรู้ และส่งบรอดแคสต์เป็นชุด ชุดละ 100 บทสนทนา

2.11.1.3 คิว typing ใช้ส่งสถานะกำลังพิมพ์ระหว่างที่แชทบอทสร้างคำตอบ

2.11.1.4 ตัวจัดตารางเวลาตรวจหาแชทที่ค้างตอบนานเกินกำหนดทุก 1 นาที

2.12 WebSocket และ Laravel Reverb

WebSocket คือโพรโทคอลที่เปิดการเชื่อมต่อค้างไว้ระหว่างผู้ใช้กับเครื่องแม่ข่าย ทำให้เครื่องแม่ข่ายส่งข้อมูลไปยังผู้ใช้ได้ทันทีโดยผู้ใช้ไม่ต้องส่งคำขอซ้ำ ดังรูปที่ 2.11 ส่วน Laravel Reverb คือเซิร์ฟเวอร์ WebSocket ของ Laravel [12] ที่ใช้โพรโทคอลเดียวกับ Pusher และรองรับช่องส่วนตัวที่ต้องขออนุญาตจากระบบหลังบ้านก่อนรับข้อมูล



รูปที่ 2.11 การเชื่อมต่อแบบ WebSocket เทียบกับ HTTP

2.12.1 ข้อดี

2.12.1.1 แสดงข้อความใหม่ได้ทันทีโดยไม่ต้องดึงข้อมูลซ้ำเป็นระยะ

2.12.1.2 ช่องส่วนตัวป้องกันไม่ให้ผู้ใช้ร้านอื่นรับข้อมูลของร้านค้าได้

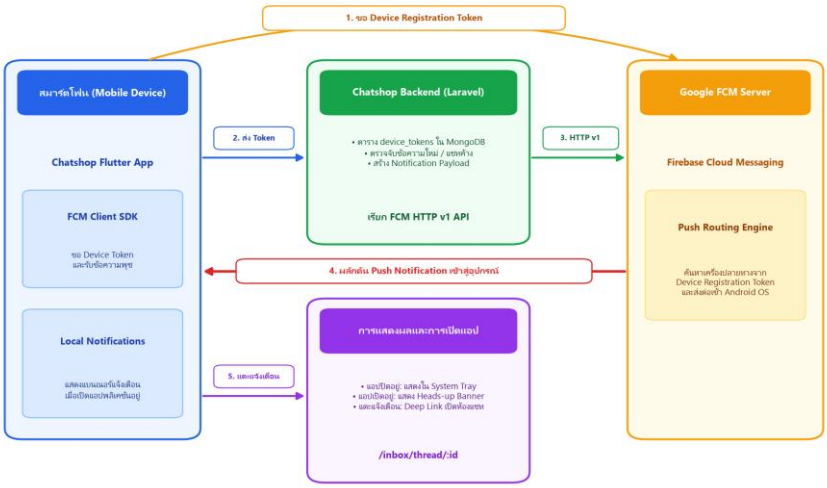
2.12.2 ข้อเสีย

2.12.2.1 การเชื่อมต่อหลุดได้เมื่อสัญญาณเครือข่ายของสมาร์ตโฟนไม่เสถียร

2.12.3 การนำไปใช้ในโครงการ ระบบหลังบ้านส่งเหตุการณ์ของร้านค้า เช่น ข้อความใหม่ การแจ้งเตือน และคำสั่งซื้อที่เปลี่ยนสถานะ ผ่านช่องส่วนตัวของแต่ละร้านค้า แอปพลิเคชันใช้แพ็กเกจ laravel_reverb เชื่อมต่อช่องนี้หลังเข้าสู่ระบบ แล้วปรับรายการบทสนทนาและหน้าต่างสนทนาที่เปิดอยู่ทันทีเมื่อได้รับเหตุการณ์

2.13 Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging คือบริการส่ง Push Notification ที่ให้บริการโดย Google [13] เครื่องแม่ข่ายส่งการแจ้งเตือนไปยังอุปกรณ์ผ่านโทเคนประจำเครื่องที่แอปพลิเคชันลงทะเบียนไว้ ดังรูปที่ 2.12 ผู้ใช้จึงได้รับการแจ้งเตือนแม้ไม่ได้เปิดแอปพลิเคชันอยู่ และแตะการแจ้งเตือนเพื่อเปิดหน้าจอที่เกี่ยวข้องได้



รูปที่ 2.12 การส่ง Push Notification ผ่าน Firebase Cloud Messaging

2.13.1 ข้อดี

2.13.1.1 ใช้งานได้โดยไม่มีค่าใช้จ่าย

2.13.1.2 ส่งการแจ้งเตือนถึงอุปกรณ์ได้แม้แอปพลิเคชันถูกปิดอยู่

2.13.2 ข้อเสีย

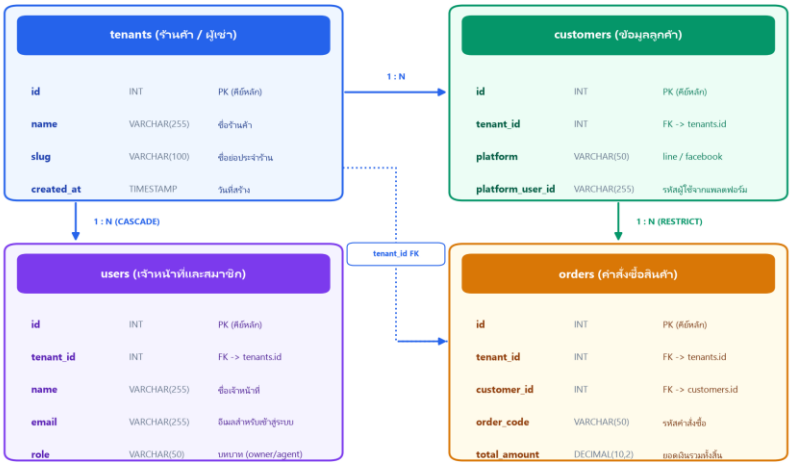
2.13.2.1 ขณะแอปพลิเคชันเปิดอยู่ ระบบปฏิบัติการจะไม่แสดงการแจ้งเตือนให้เอง แอปพลิเคชันต้องจัดการแสดงผลเอง

2.13.3 การนำไปใช้ในโครงการ แอปพลิเคชันลงทะเบียนโทเคนของอุปกรณ์เมื่อเข้าสู่ระบบ ปรับปรุงโทเคนเมื่อมีการเปลี่ยนแปลง และยกเลิกเมื่อออกจากระบบ ส่วนระบบหลังบ้านจะส่งการแจ้งเตือนเมื่อมีข้อความใหม่ แชทค้างตอบนานเกินกำหนด หรือแชทบอทสร้างคำตอบไม่สำเร็จ หากแอปพลิเคชันเปิดอยู่จะแสดงการแจ้งเตือนด้วยแพ็กเกจ flutter_local_notifications แทน เมื่อผู้ใช้แตะการแจ้งเตือน แอปพลิเคชันจะเปิดหน้าต่างสนทนานั้นทันที

2.14 MySQL

MySQL คือระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์ส [14] จัดเก็บข้อมูลในรูปตารางที่ประกอบด้วยแถวและคอลัมน์ แต่ละตารางมีคีย์หลักสำหรับระบุแถวที่ไม่ซ้ำกัน และใช้คีย์นอกอ้างอิงแถวของตารางอื่นเพื่อบังคับความสัมพันธ์ของข้อมูล ดังรูปที่ 2.13 ผู้ใช้จัดการข้อมูลด้วยภาษา SQL ระบบรองรับธุรกรรมตามคุณสมบัติ ACID ที่ทำให้กลุ่มคำสั่งสำเร็จทั้งหมดหรือไม่สำเร็จเลย และ

สร้างดัชนีเพื่อเพิ่มความเร็วในการค้นหาได้ นอกจากนี้ MySQL ยังมีชนิดข้อมูล JSON สำหรับเก็บข้อมูลที่เป็นรายการไว้ในคอลัมน์เดียว



รูปที่ 2.13 ตัวอย่างตารางที่เชื่อมกันด้วยคีย์หลักและคีย์นอก

2.14.1 ข้อดี

2.14.1.1 รักษาความถูกต้องของข้อมูลที่สัมพันธ์กันด้วยคีย์หลักและคีย์นอก

2.14.1.2 รองรับธุรกรรม เหมาะกับข้อมูลที่ต้องการความถูกต้องสูง

2.14.1.3 รวมข้อมูลจากหลายตารางในคำสั่งเดียวได้ด้วยการเชื่อมตาราง

2.14.2 ข้อเสีย

2.14.2.1 ต้องกำหนดโครงสร้างตารางล่วงหน้า ไม่เหมาะกับข้อมูลที่เปลี่ยนบ่อย

2.14.2.2 การเพิ่มคอลัมน์ในตารางที่มีข้อมูลจำนวนมากใช้เวลานาน

2.14.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ MySQL จัดเก็บข้อมูลที่มีความสัมพันธ์กันชัดเจนและต้องการความถูกต้องสูง ดังนี้

2.14.3.1 ข้อมูลร้านค้าและผู้ใช้ ได้แก่ ร้านค้า สมาชิก บทบาท และคำเชิญ

2.14.3.2 ข้อมูลธุรกิจและช่องทาง ได้แก่ ธุรกิจ ช่องทางที่เชื่อมต่อ และการตั้งค่าแชทบอทของแต่ละธุรกิจ

2.14.3.3 ข้อมูลการขาย ได้แก่ ลูกค้า สินค้า และคำสั่งซื้อ

2.14.3.4 ข้อมูลสนับสนุนการตอบแชท ได้แก่ แท็ก ข้อความสำเร็จรูป กฎตอบกลับอัตโนมัติ ผู้รับผิดชอบบทสนทนา และประวัติการนำเข้าแชท

2.14.4 แนวทางการออกแบบตารางในโครงการ ดังนี้

2.14.4.1 ทุกตารางที่เป็นข้อมูลของร้านค้าอ้างอิงตารางร้านค้าด้วยคีย์นอกแบบลบตามกัน เมื่อร้านค้าถูกลบ ข้อมูลที่เกี่ยวข้องจะถูกลบตามโดยอัตโนมัติ

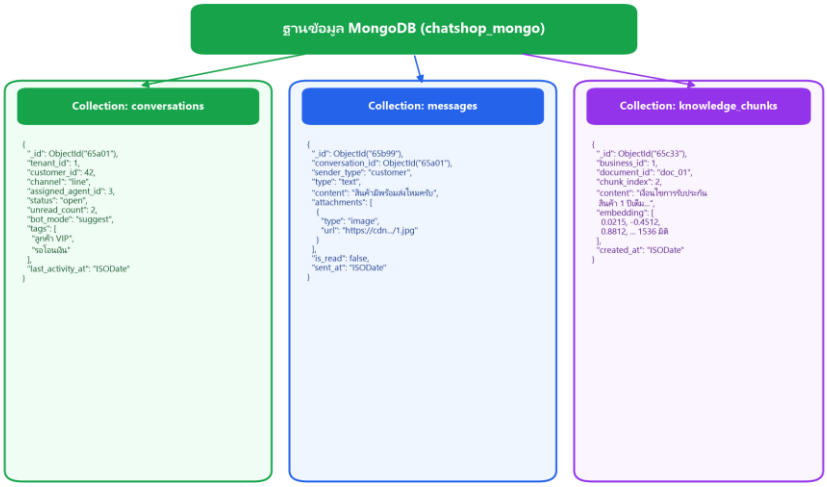
2.14.4.2 เก็บข้อมูลที่เป็นรายการ เช่น สิทธิ์ของบทบาทและคีย์เวิร์ดของกฎตอบกลับ ไว้ในคอลัมน์ชนิด JSON

2.14.4.3 ใช้ชุดอักขระ utf8mb4 ซึ่งรองรับทั้งภาษาไทยและอีโมจิ

2.14.4.4 สร้างและแก้ไขโครงสร้างตารางผ่าน Migration ของ Laravel จึงติดตามประวัติการเปลี่ยนแปลงโครงสร้างได้

2.15 MongoDB

MongoDB คือฐานข้อมูลแบบเอกสาร จัดเก็บข้อมูลเป็นเอกสารรูปแบบ BSON ซึ่งมีโครงสร้างคล้าย JSON [15] และรวมเอกสารประเภทเดียวกันไว้ในคอลเลกชัน เอกสารในคอลเลกชันเดียวกันไม่จำเป็นต้องมีฟิลด์เหมือนกัน และเก็บอาร์เรย์หรือเอกสารย่อยไว้ในเอกสารได้โดยตรง ดังรูปที่ 2.14 เอกสารทุกฉบับมีรหัส ObjectId ที่ระบบสร้างให้เป็นคีย์หลัก การใช้ฐานข้อมูลหลายประเภทในระบบเดียวตามลักษณะของข้อมูลเรียกว่า Polyglot Persistence [16]



รูปที่ 2.14 โครงสร้างฐานข้อมูล คอลเลกชัน และเอกสารของ MongoDB

2.15.1 ข้อดี

2.15.1.1 โครงสร้างข้อมูลยืดหยุ่น เพิ่มฟิลด์ได้โดยไม่ต้องแก้โครงสร้างตาราง

2.15.1.2 รองรับข้อมูลปริมาณมากและเก็บอาร์เรย์ในเอกสารได้โดยตรง

2.15.1.3 อ่านข้อมูลที่เกี่ยวข้องได้จากเอกสารเดียวโดยไม่ต้องเชื่อมหลายตาราง

2.15.2 ข้อเสีย

2.15.2.1 ไม่มีการบังคับความสัมพันธ์ระหว่างข้อมูลเหมือนฐานข้อมูลเชิงสัมพันธ์

2.15.2.2 ผู้พัฒนาต้องตรวจสอบความถูกต้องของฟิลด์ในโปรแกรมเอง

2.15.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ MongoDB เก็บข้อมูลที่มีปริมาณมาก เพิ่มขึ้นตลอดเวลา หรือมีรูปแบบไม่แน่นอน แบ่งเป็น 5 คอลเลกชัน ดังนี้

2.15.3.1 conversations เก็บบทสนทนาของลูกค้าแต่ละรายในแต่ละช่องทาง พร้อมแท็ก ผู้รับผิดชอบ และโหมดแชทบอท

2.15.3.2 messages เก็บข้อความทุกข้อความ ซึ่งมีฟิลด์ต่างกันตามประเภท เช่น ข้อความสติกเกอร์มีรหัสสติกเกอร์ และข้อความไฟล์มีชื่อและขนาดไฟล์

2.15.3.3 knowledge_documents เก็บเอกสารความรู้ของแชทบอทแต่ละธุรกิจ

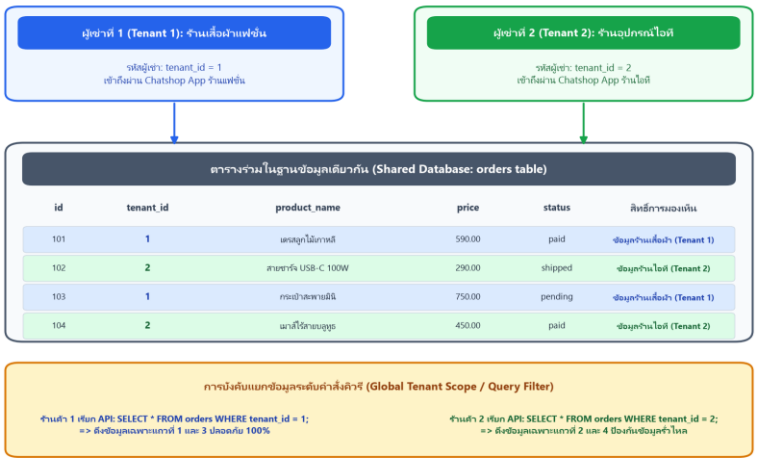
2.15.3.4 knowledge_chunks เก็บส่วนย่อยของเอกสารพร้อมเวกเตอร์

2.15.3.5 device_tokens เก็บโทเคนของอุปกรณ์สำหรับส่งการแจ้งเตือน

2.15.4 การเชื่อมต่อกับระบบหลังบ้าน ระบบหลังบ้านเชื่อมต่อ MongoDB ผ่านไลบรารี laravel-mongodb ซึ่งใช้ Model รูปแบบเดียวกับ Eloquent และเชื่อมโยงข้อมูลกับ MySQL ด้วยรหัสอ้างอิง เช่น เอกสารบทสนทนาเก็บรหัสร้านค้าและรหัสลูกค้าจาก MySQL ส่วนตารางคำสั่งซื้อเก็บรหัสเอกสารบทสนทนาจาก MongoDB

2.16 สถาปัตยกรรม Multi-Tenant

สถาปัตยกรรม Multi-Tenant คือการออกแบบให้ระบบเดียวให้บริการผู้เช่าหลายรายโดยข้อมูลของผู้เช่าแต่ละรายแยกออกจากกัน [17] วิธีที่ใช้กันทั่วไปคือใช้ฐานข้อมูลร่วมกันและเพิ่มคอลัมน์รหัสผู้เช่าในทุกตาราง แล้วกรองข้อมูลตามรหัสผู้เช่าทุกครั้งที่อ่านหรือเขียน ดังรูปที่ 2.15

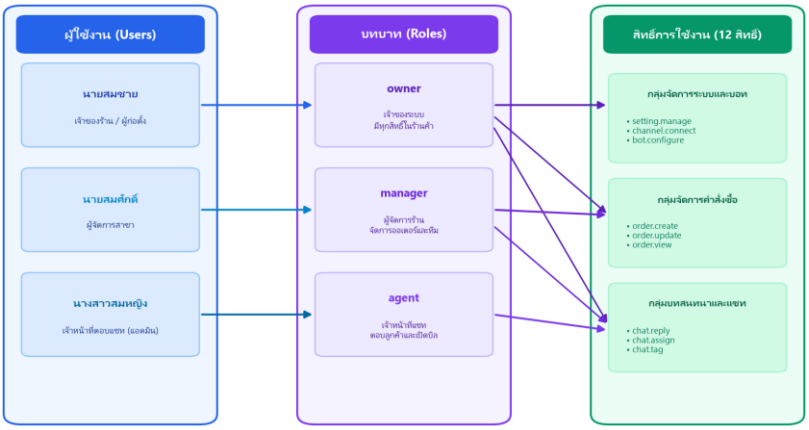


รูปที่ 2.15 การแยกข้อมูลของผู้เช่าด้วยคอลัมน์รหัสผู้เช่า

2.16.1 การนำไปใช้ในโครงการ ทุกตารางและทุกเอกสารมีรหัสร้านค้ากำกับ ร้านค้าหนึ่งแห่งมีได้หลายธุรกิจ แต่ละธุรกิจมีช่องทาง แบรนด์ และการตั้งค่าแชทบอทของตนเอง

2.17 การควบคุมสิทธิ์ตามบทบาท (RBAC)

RBAC (Role-Based Access Control) คือการควบคุมการเข้าถึงระบบโดยกำหนดสิทธิ์ให้แก่บทบาท แล้วมอบบทบาทให้ผู้ใช้ [18] ดังรูปที่ 2.16 เมื่อต้องการเปลี่ยนสิทธิ์ของผู้ใช้หลายคน ผู้ดูแลแก้ไขเฉพาะบทบาทโดยไม่ต้องแก้ทีละคน ช่วยลดความผิดพลาดในการกำหนดสิทธิ์



รูปที่ 2.16 ความสัมพันธ์ระหว่างผู้ใช้ บทบาท และสิทธิ์

2.17.1 การนำไปใช้ในโครงการ ระบบมีบทบาทพื้นฐานที่กำหนดไว้ล่วงหน้า 5 บทบาท ได้แก่ owner, manager, admin, agent และ viewer และเจ้าของร้านสร้างบทบาทใหม่ได้ โดยเลือกจากสิทธิ์ทั้งหมด 12 รายการ เช่น การตอบข้อความ การมอบหมายงาน และการจัดการคำสั่งซื้อ โดยระบบหลังบ้านตรวจสอบสิทธิ์ทุกครั้งที่มีคำขอ

2.18 แชทบอท

แชทบอทคือโปรแกรมที่สนทนากับผู้ใช้ด้วยภาษาธรรมชาติผ่านช่องทางแชท [19] แชทบอทแบบกฎจะตอบตามคำสำคัญหรือเงื่อนไขที่กำหนดไว้ล่วงหน้า ส่วนแชทบอทที่ใช้แบบจำลองภาษาสามารถสร้างคำตอบใหม่ได้ตามบริบทของการสนทนา แต่อาจสร้างข้อมูลที่ไม่มีอยู่จริงได้ ร้านค้าจึงต้องให้แชทบอทส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อตอบคำถามไม่ได้

2.18.1 การนำไปใช้ในโครงการ แชทบอทตอบลูกค้าได้ทุกช่องทาง ร้านค้ากำหนดโหมดเริ่มต้นของแต่ละธุรกิจ และเจ้าหน้าที่เปลี่ยนโหมดรายบทสนทนาได้จากแอปพลิเคชัน ดังนี้

2.18.1.1 Manual เจ้าหน้าที่ตอบลูกค้าเองทั้งหมด

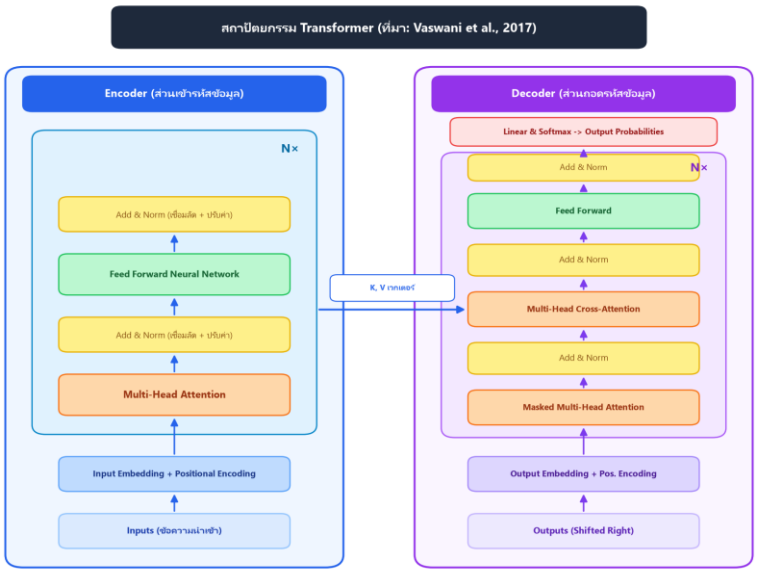
2.18.1.2 Suggest แชทบอทเสนอคำตอบให้เจ้าหน้าที่เลือกใช้

2.18.1.3 Autopilot แชทบอทตอบลูกค้าเอง ส่งต่อให้เจ้าหน้าที่เมื่อตอบไม่ได้

2.18.2 การตอบอัตโนมัติด้วยคีย์เวิร์ด นอกจากแชทบอทที่ใช้แบบจำลองภาษา ระบบมีการตอบอัตโนมัติแบบกฎ ร้านค้ากำหนดคีย์เวิร์ดและข้อความตอบกลับได้เอง เมื่อข้อความของลูกค้าตรงกับคีย์เวิร์ดทั้งข้อความหรือมีคีย์เวิร์ดอยู่ ระบบจะส่งข้อความที่กำหนดไว้ทันที

2.19 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

แบบจำลองภาษาขนาดใหญ่คือแบบจำลองที่ผ่านการฝึกด้วยข้อความจำนวนมหาศาลเพื่อทำนายคำถัดไปของข้อความ แบบจำลองส่วนใหญ่ใช้สถาปัตยกรรม Transformer ซึ่งใช้กลไก Self-Attention พิจารณาความสัมพันธ์ระหว่างคำทุกคำในข้อความพร้อมกัน [20] ดังรูปที่ 2.17 เมื่อแบบจำลองมีขนาดใหญ่เพียงพอ จะสามารถทำงานใหม่ได้จากคำสั่งและตัวอย่างที่ให้ไว้ในข้อความนำเข้า โดยไม่ต้องฝึกแบบจำลองเพิ่มเติม [21]

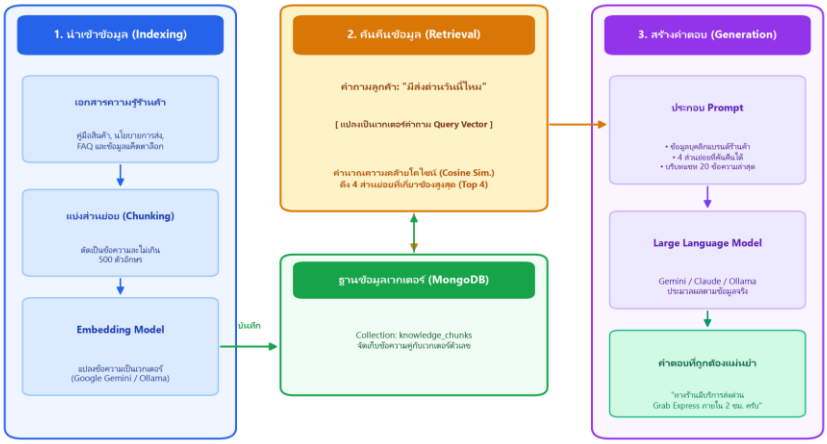


รูปที่ 2.17 สถาปัตยกรรม Transformer (ที่มา: [20])

2.19.1 การนำไปใช้ในโครงการ ระบบหลังบ้านประกอบข้อความสั่งงานจากบุคลิกที่ร้านค้ากำหนด ข้อมูลแบรนด์ ผลการค้นคืนจากฐานความรู้ บทสรุปของบทสนทนา และข้อความล่าสุด 20 ข้อความ แล้วส่งให้แบบจำลองภาษาสร้างคำตอบ ร้านค้าเลือกใช้ Ollama ที่ติดตั้งบนเครื่องแม่ข่ายเอง หรือบริการผ่าน API เช่น Anthropic Claude และ Google Gemini ได้ นอกจากนี้แอปพลิเคชันยังเรียกให้แบบจำลองภาษาวิเคราะห์บทสนทนา เพื่อแสดงบทสรุป ความรู้สึก และเจตนาของลูกค้าแก่เจ้าหน้าที่

2.20 RAG (Retrieval-Augmented Generation)

RAG คือเทคนิคที่ค้นคืนเอกสารที่เกี่ยวข้องกับคำถามจากฐานความรู้ก่อน แล้วส่งเอกสารเหล่านั้นให้แบบจำลองภาษาใช้ประกอบการสร้างคำตอบ [22] ดังรูปที่ 2.18 ช่วยให้แบบจำลองตอบจากข้อมูลของร้านค้าที่ไม่ได้อยู่ในข้อมูลที่ใช้ฝึก และลดการสร้างข้อมูลที่ไม่มีอยู่จริง

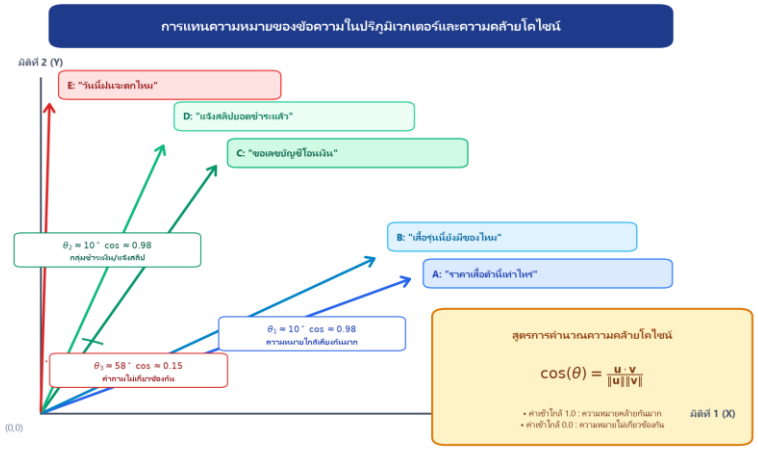


รูปที่ 2.18 ขั้นตอนการทำงานของ RAG

2.20.1 การนำไปใช้ในโครงการ ร้านค้าเพิ่มเอกสารความรู้และนำข้อมูลสินค้าเข้าสู่ฐานความรู้ได้ ระบบแบ่งเอกสารเป็นส่วนย่อยขนาดไม่เกิน 500 ตัวอักษร จัดเก็บพร้อมเวกเตอร์ใน MongoDB และค้นคืนส่วนย่อยที่เกี่ยวข้อง 4 อันดับแรกมาประกอบข้อความสั่งงาน

2.21 Embedding และความคล้ายโคไซน์

Embedding คือการแปลงข้อความเป็นเวกเตอร์ตัวเลขที่มีมิติคงที่ โดยข้อความที่มีความหมายใกล้เคียงกันจะได้เวกเตอร์ที่อยู่ใกล้กันในปริภูมิเวกเตอร์ [23] ดังรูปที่ 2.19 ความใกล้เคียงวัดได้จากความคล้ายโคไซน์ $\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$ ค่าที่เข้าใกล้ 1 แสดงว่าข้อความทั้งสองมีความหมายใกล้เคียงกัน ส่วนค่าที่เข้าใกล้ 0 แสดงว่าไม่เกี่ยวข้องกัน



รูปที่ 2.19 ตัวอย่างตำแหน่งของข้อความในปริภูมิเวกเตอร์

2.21.1 การนำไปใช้ในโครงการ ระบบแปลงเอกสารความรู้และคำถามของลูกค้าเป็นเวกเตอร์ด้วยแบบจำลอง Embedding ของ Google Gemini หรือ Ollama แล้วคำนวณความคล้ายโคไซน์ระหว่างคำถามกับทุกส่วนย่อย และเลือกเฉพาะส่วนย่อยที่มีค่าตั้งแต่ 0.3 ขึ้นไป

2.22 งานวิจัยที่เกี่ยวข้อง

จากการศึกษาแนวคิด ทฤษฎี และเทคโนโลยีที่นำมาใช้ในการพัฒนาระบบ ผู้จัดทำได้ศึกษาและรวบรวมงานวิจัยรวมถึงวิทยานิพนธ์ที่เกี่ยวข้องกับการพัฒนาระบบรวมศูนย์ข้อความ การประยุกต์ใช้แชทบอทเพื่อสนับสนุนการสื่อสารและการพาณิชย์ ตลอดจนระบบจัดการคำสั่งซื้อ เพื่อนำสาระสำคัญและข้อจำกัดของงานวิจัยดังกล่าวมาเป็นแนวทางในการออกแบบและพัฒนาแอปพลิเคชัน Chatshop โดยมีรายละเอียดดังต่อไปนี้

ฑัณฑิกา นันยากรสกุล [24] ได้ศึกษาระบบแช็ตบอตและการวิเคราะห์ข้อมูลขั้นสูงของธุรกิจบริการจัดส่งพัสดุไปต่างประเทศ โดยพัฒนาระบบสื่อสารกับลูกค้าผ่าน LINE Messaging API เพื่อช่วยแนะนำข้อมูลและตรวจสอบสถานะการจัดส่งพัสดุอัตโนมัติตลอดเวลา พร้อมทั้งนำข้อมูลประวัติการสนทนามาประมวลผลเพื่อวิเคราะห์พฤติกรรมและความต้องการของลูกค้า รวมถึงมีกลไกส่งต่อข้อความแจ้งเตือนแก่เจ้าหน้าที่เมื่อบทสนทนามีความซับซ้อนเกินกว่าระบบอัตโนมัติจะประมวลผลได้ งานวิจัยดังกล่าวช่วยลดภาระงานของเจ้าหน้าที่และลดระยะเวลาการตอบกลับลูกค้าได้อย่างมีประสิทธิภาพ อย่างไรก็ตาม ระบบยังรองรับการสื่อสารผ่านช่องทาง LINE เพียงช่องทางเดียว และการทำงานของเจ้าหน้าที่ยังไม่มีระบบรวมศูนย์บนสมาร์ตโฟนที่สามารถจัดการข้อความและดำเนินธุรกรรมคำสั่งซื้อได้พร้อมกันในจุดเดียว

ปาลิตา อนันตนิติเวทย์ [25] ได้ศึกษาการพัฒนาเว็บแอปพลิเคชันและแช็ตบอตของธุรกิจขายเสื้อผ้า โดยพัฒนาระบบให้สามารถโต้ตอบกับลูกค้าผ่านแอปพลิเคชัน LINE ด้วย LINE Messaging API เพื่อตอบคำถาม ให้ข้อมูลสินค้า และตรวจสอบสถานะรายการสั่งซื้อ พร้อมทั้งพัฒนาเว็บแอปพลิเคชันสำหรับเจ้าหน้าที่ร้านค้าเพื่อใช้จัดการรายการสินค้า ตรวจสอบคำสั่งซื้อ และจัดการสินค้าคงคลัง โดยมีการกำหนดสิทธิ์การเข้าถึงข้อมูลตามบทบาทของผู้ใช้งาน ผลการวิจัยพบว่าระบบช่วยเพิ่มความสะดวกรวดเร็วในการตอบสนองความต้องการของลูกค้าและช่วยให้เจ้าหน้าที่ทำงานได้อย่างเป็นระบบมากยิ่งขึ้น อย่างไรก็ตาม ระบบดังกล่าวยังรองรับเฉพาะแพลตฟอร์ม LINE เท่านั้น อีกทั้งการจัดการคำสั่งซื้อยังทำงานแยกส่วนอยู่บนหน้าเว็บแอปพลิเคชัน จึงยังไม่รองรับการจัดการคำสั่งซื้อได้โดยตรงจากหน้าต่างสนทนาบนสมาร์ตโฟนของเจ้าหน้าที่

เสริมศิริ นวลลิขิต [26] ได้ศึกษาการพัฒนาระบบแชตบอตสำหรับธุรกิจจำหน่ายบรรจุภัณฑ์ไปรษณีย์ โดยพัฒนาระบบการตอบกลับข้อความและอำนวยความสะดวกในการสั่งซื้อสินค้าผ่านแอปพลิเคชัน LINE ซึ่งนำเทคโนโลยีการประมวลผลภาษาธรรมชาติและจำแนกเจตนาของข้อความมาประยุกต์ใช้เพื่อแนะนำรายการสินค้า คำนวณราคา และรับคำสั่งซื้อจากลูกค้าผ่านบทสนทนาโดยตรง ช่วยให้กระบวนการซื้อขายดำเนินไปได้อย่างรวดเร็วและต่อเนื่อง ผลการประเมินพบว่าระบบช่วยลดเวลาในการให้บริการและสร้างความพึงพอใจให้แก่ลูกค้าในระดับสูง อย่างไรก็ตาม ระบบดังกล่าวเน้นการโต้ตอบแบบอัตโนมัติผ่านช่องทาง LINE เพียงช่องทางเดียว และยังขาดระบบรองรับการสลับการทำงานมายังเจ้าหน้าที่ร้านค้าสำหรับการสนทนาข้ามหลายแพลตฟอร์มอย่างเป็นระบบ

Sanchez [27] ได้ศึกษาวิจัยเรื่อง Facebook Messenger chatbots: How businesses can benefit from this opportunity โดยศึกษาการประยุกต์ใช้ระบบสนทนาอัตโนมัติบน Facebook Messenger Platform เพื่อสนับสนุนการดำเนินธุรกิจการค้าและการสร้างปฏิสัมพันธ์กับลูกค้า งานวิจัยได้วิเคราะห์สถาปัตยกรรมการรับส่งข้อความ การเชื่อมต่อผ่าน Webhook และการจัดการธุรกรรมซื้อขายผ่านบทสนทนา พร้อมทั้งชี้ให้เห็นถึงความจำเป็นในการผสานการทำงานระหว่างระบบอัตโนมัติกับเจ้าหน้าที่ เพื่อให้เจ้าหน้าที่สามารถเข้ามารับช่วงต่อบทสนทนาในกรณีที่ลูกค้ามีข้อซักถามเฉพาะด้านหรือต้องการปิดการขายที่ซับซ้อนได้อย่างราบรื่น อย่างไรก็ตาม งานวิจัยดังกล่าวมุ่งเน้นการศึกษาเฉพาะบนแพลตฟอร์ม Facebook Messenger เพียงช่องทางเดียว และยังไม่มีการพัฒนาแอปพลิเคชันเพื่อรวมศูนย์การสนทนาข้ามแพลตฟอร์มสำหรับเจ้าหน้าที่ร้านค้า

จากการศึกษางานวิจัยและวิทยานิพนธ์ที่เกี่ยวข้องข้างต้น พบว่าการนำระบบสนทนาอัตโนมัติมาประยุกต์ใช้ในธุรกิจช่วยเพิ่มประสิทธิภาพในการตอบสนองลูกค้าและสนับสนุนการสร้างคำสั่งซื้อได้อย่างมีประสิทธิผล แต่งานวิจัยส่วนใหญ่ยังคงจำกัดการทำงานอยู่บนช่องทางใดช่องทางหนึ่งเพียงช่องทางเดียว เช่น LINE หรือ Facebook และระบบจัดการคำสั่งซื้อส่วนใหญ่ยังทำงานแยกส่วน

จากระบบสนทนา หรือต้องดำเนินการผ่านหน้าเว็บแอปพลิเคชันเท่านั้น ผู้จัดทำจึงนำข้อค้นพบและข้อจำกัดดังกล่าวมาเป็นแนวทางในการพัฒนาแอปพลิเคชัน Chatshop โดยออกแบบสถาปัตยกรรมระบบที่รวมศูนย์ข้อความจากหลากหลายแพลตฟอร์ม ได้แก่ LINE Facebook Messenger Instagram และ WebChat เข้าสู่ระบบหลังบ้านเดียวกัน พร้อมทั้งพัฒนาแอปพลิเคชันสำหรับสมาร์ตโฟนด้วย Flutter ที่ช่วยให้เจ้าหน้าที่สามารถจัดการบทสนทนา เปิดคำสั่งซื้อสินค้าได้ทันทีจากหน้าต่างสนทนา และควบคุมโหมดการทำงานของแชทบอทร่วมกับเทคนิค RAG ได้อย่างมีประสิทธิภาพ

บทที่ 3
วิธีดำเนินงาน

ผู้จัดทำพัฒนา Chatshop เป็น 2 ส่วน ได้แก่ แอปพลิเคชันบนสมาร์ตโฟนที่เจ้าหน้าที่ร้านค้าใช้งาน และระบบบริการหลังบ้านที่รับส่งข้อความกับแพลตฟอร์มภายนอก จัดเก็บข้อมูล และประมวลผลแชทบอท บทนี้อธิบายขั้นตอนการดำเนินงานตั้งแต่การวิเคราะห์ปัญหาจนถึงการพัฒนาทั้งสองส่วน โดยมีหัวข้อดังนี้

3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน

3.2 การวิเคราะห์ระบบ

3.3 การออกแบบระบบ

3.4 โครงสร้างแอปพลิเคชันบนสมาร์ตโฟน

3.5 โครงสร้างระบบบริการหลังบ้าน

3.6 การออกแบบ API

3.7 การพัฒนาแอปพลิเคชันบนสมาร์ตโฟน

3.8 การตั้งค่าแพลตฟอร์มภายนอก

3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน

ร้านค้าออนไลน์ที่ขายผ่านแชทต้องเปิดแอปพลิเคชันของแต่ละช่องทางแยกกัน เจ้าหน้าที่ต้องสลับหน้าจอไปมาจนพลาดการแจ้งเตือน เมื่อมีหลายคนช่วยกันตอบก็ตอบซ้ำกันหรือตกหล่น คำถามเดิมต้องพิมพ์ตอบซ้ำทุกวัน และการเปิดคำสั่งซื้อยังจดด้วยมือแยกจากแชท ผู้จัดทำจึงพัฒนาระบบหลังบ้านที่รับข้อความจากทุกช่องทางมาเก็บไว้ที่เดียว และพัฒนาแอปพลิเคชันที่แสดงข้อความเหล่านั้นในกล่องข้อความเดียวแบบเรียลไทม์ พร้อม Push Notification แชทบอท และการเปิดคำสั่งซื้อจากหน้าต่างสนทนา

3.2 การวิเคราะห์ระบบ

ผู้จัดทำวิเคราะห์ระบบจากขั้นตอนการทำงานจริงของร้านค้าที่ขายผ่านแชท แล้วสรุปเป็นผู้ใช้งาน ความต้องการด้านการทำงาน และความต้องการด้านคุณภาพของระบบ ดังนี้

3.2.1 ผู้ใช้งานของระบบ แบ่งออกเป็น 3 กลุ่ม ดังนี้

3.2.1.1 เจ้าของร้าน ผู้สมัครใช้งาน เชื่อมต่อช่องทางแชท เชิญสมาชิกเข้าทีม กำหนดบทบาท และตั้งค่าร้านค้าและแชทบอทผ่านแอปพลิเคชัน

3.2.1.2 เจ้าหน้าที่ ได้แก่ ผู้จัดการ แอดมิน และพนักงานตอบแชท ใช้งานได้ตามสิทธิ์ของบทบาท เช่น ตอบแชท เปิดคำสั่งซื้อ หรือดูรายงาน

3.2.1.3 ลูกค้า ผู้ส่งข้อความหาร้านค้าโดยไม่ต้องติดตั้งแอปพลิเคชันของร้าน ผ่านช่องทาง LINE, Facebook Messenger, Instagram หรือ WebChat

3.2.2 Functional Requirements แบ่งตามส่วนของระบบ ดังนี้

3.2.2.1 ส่วนแอปพลิเคชันบนสมาร์ตโฟน

1) ผู้ใช้ต้องสมัครและเข้าสู่ระบบด้วยอีเมล Facebook หรือ Google ได้

2) เจ้าหน้าที่ต้องดูบทสนทนาจากทุกช่องทางในกล่องข้อความเดียว เลือกดูตามช่องทาง เพจ ข้อความที่ยังไม่อ่าน และค้นหาได้

3) เจ้าหน้าที่ต้องตอบข้อความ ส่งรูปภาพ วิดีโอ และไฟล์เอกสาร ใช้ข้อความสำเร็จรูป และตอบกลับโดยอ้างอิงข้อความเดิมได้

4) แอปพลิเคชันต้องแสดงข้อความใหม่ทันทีโดยผู้ใช้ไม่ต้องรีเฟรช และส่ง Push Notification เมื่อไม่ได้เปิดแอปพลิเคชันอยู่

5) เจ้าหน้าที่ต้องมอบหมาย ปักหมุด จัดเก็บ และติดแท็กบทสนทนาได้

6) เจ้าหน้าที่ต้องเปลี่ยนโหมดแชทบอทของแต่ละบทสนทนา ขอคำตอบแนะนำ และดูผลวิเคราะห์บทสนทนาได้

7) เจ้าหน้าที่ต้องจัดการข้อมูลลูกค้าและสินค้า และเปิดคำสั่งซื้อจากหน้าต่างสนทนาได้

8) เจ้าของร้านต้องจัดการทีม ช่องทาง และการตั้งค่าร้านค้าได้

9) ผู้ใช้ต้องดูรายงานภาพรวมการให้บริการได้

3.2.2.2 ส่วนระบบบริการหลังบ้าน

1) ระบบต้องรับข้อความจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ผ่าน Webhook

2) ระบบต้องส่งข้อความที่เจ้าหน้าที่ตอบกลับออกไปยังแพลตฟอร์มของลูกค้าได้ถูกต้อง

3) ระบบต้องส่งเหตุการณ์ไปยังแอปพลิเคชันแบบเรียลไทม์ และส่งการแจ้งเตือนผ่าน Firebase Cloud Messaging

4) ระบบต้องแจ้งเตือนเจ้าหน้าที่เมื่อมีแชทค้างตอบนานเกินกำหนด

5) ระบบต้องสร้างคำตอบด้วยแชทบอทจากฐานความรู้ของร้าน และโอนให้เจ้าหน้าที่เมื่อตอบไม่ได้

6) ระบบต้องตอบกลับอัตโนมัติตามคีย์เวิร์ดที่ร้านกำหนด

7) ระบบต้องเปิด REST API ให้แอปพลิเคชัน และตรวจสิทธิ์ทุกคำขอ

3.2.3 Non-Functional Requirements เป็นความต้องการด้านคุณภาพของระบบ ดังนี้

3.2.3.1 ด้านประสิทธิภาพ ระบบหลังบ้านต้องตอบกลับ Webhook ได้ทันที แล้วจึงประมวลผลข้อความในคิวงานเบื้องหลัง เพื่อไม่ให้แพลตฟอร์มส่ง Webhook ซ้ำ

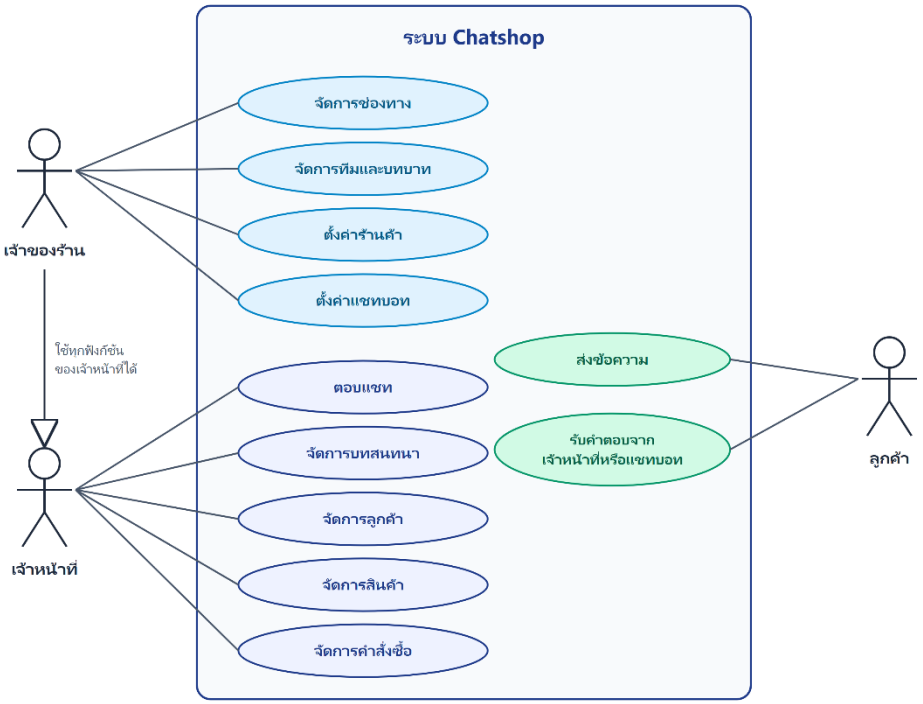
3.2.3.2 ด้านความปลอดภัย ระบบต้องตรวจลายเซ็นของ Webhook ทุกครั้ง ยืนยันตัวตนด้วยโทเคนก่อนเรียกใช้ API และแอปพลิเคชันต้องเก็บโทเคนในพื้นที่จัดเก็บที่เข้ารหัส

3.2.3.3 ด้านการแยกข้อมูล ข้อมูลของแต่ละร้านค้าต้องแยกออกจากกัน ผู้ใช้เห็นได้เฉพาะข้อมูลของร้านตนเอง แม้ทุกร้านจะใช้ฐานข้อมูลชุดเดียวกัน

3.2.3.4 ด้านการใช้งาน แอปพลิเคชันต้องแสดงผลการกระทำของผู้ใช้ทันที เช่น การส่งข้อความหรือการมอบหมายงาน และย้อนกลับสถานะเดิมเมื่อเครื่องแม่ข่ายตอบว่าไม่สำเร็จ

3.2.3.5 ด้านความน่าเชื่อถือ ระบบต้องลองส่งงานใหม่เมื่อล้มเหลว ต้องไม่ส่งข้อความเดิมซ้ำให้ลูกค้า และแอปพลิเคชันต้องดึงข้อมูลล่าสุดใหม่ทุกครั้งที่เชื่อมต่อเรียลไทม์กลับมาได้

3.2.4 Use Case Diagram แสดงขอบเขตของระบบและความสัมพันธ์ระหว่างผู้ใช้งานกับฟังก์ชันของระบบ ดังรูปที่ 3.1



รูปที่ 3.1 Use Case Diagram ของระบบ Chatshop

จากรูปที่ 3.1 ผู้ใช้งานแต่ละกลุ่มทำงานได้ ดังนี้

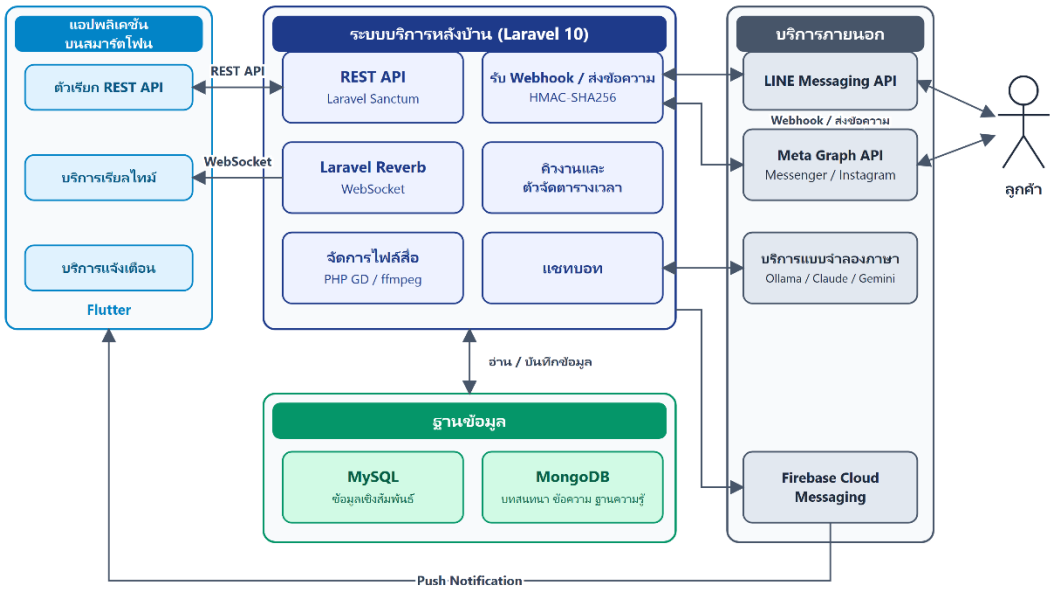
3.2.4.1 เจ้าของร้าน จัดการช่องทาง ทีม บทบาท การตั้งค่าร้าน และแชทบอท และใช้งานทุกฟังก์ชันของเจ้าหน้าที่ได้โดยไม่ต้องกำหนดสิทธิ์เพิ่ม

3.2.4.2 เจ้าหน้าที่ ตอบแชท จัดการบทสนทนา ลูกค้า สินค้า และคำสั่งซื้อตามสิทธิ์ของบทบาท

3.2.4.3 ลูกค้า ส่งข้อความและรับคำตอบจากเจ้าหน้าที่หรือแชทบอท

3.3 การออกแบบระบบ

3.3.1 สถาปัตยกรรมของระบบ แบ่งการทำงานออกเป็น 4 ส่วน ดังรูปที่ 3.2



รูปที่ 3.2 สถาปัตยกรรมของระบบ Chatshop

จากรูปที่ 3.2 ส่วนประกอบของระบบทำงาน ดังนี้

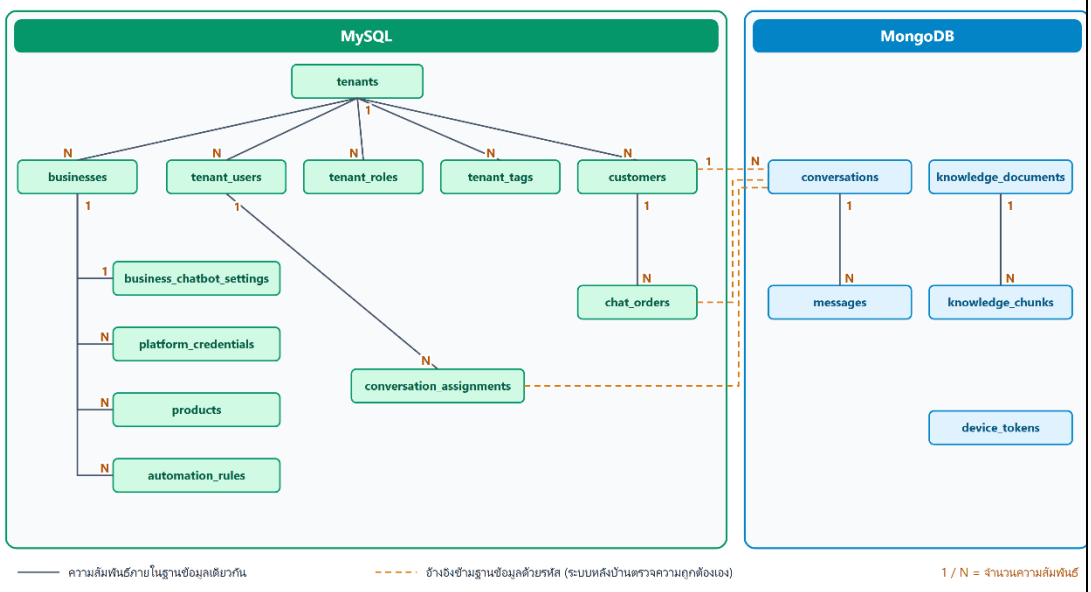
3.3.1.1 แอปพลิเคชันบนสมาร์ตโฟน พัฒนาด้วย Flutter เรียกข้อมูลผ่าน REST API รับเหตุการณ์ผ่าน WebSocket และรับการแจ้งเตือนผ่าน Firebase Cloud Messaging

3.3.1.2 ระบบบริการหลังบ้าน พัฒนาด้วย Laravel 10 รับ Webhook จากแพลตฟอร์ม ประมวลผลข้อมูลในคิวงาน สร้างคำตอบของแชทบอท และเปิด REST API ให้แอปพลิเคชัน

3.3.1.3 ฐานข้อมูล ใช้ MySQL เก็บข้อมูลเชิงสัมพันธ์ ส่วนบทสนทนา ข้อความ ฐานความรู้ และโทเคนของอุปกรณ์เก็บใน MongoDB

3.3.1.4 บริการภายนอก ได้แก่ LINE Messaging API, Meta Graph API, บริการแบบจำลองภาษา และ Firebase Cloud Messaging

3.3.2 การออกแบบฐานข้อมูล ระบบแบ่งข้อมูลไว้ใน 2 ฐานข้อมูลตามลักษณะของข้อมูล แสดงความสัมพันธ์ดังรูปที่ 3.3



รูปที่ 3.3 ความสัมพันธ์ของข้อมูลใน MySQL และ MongoDB

จากรูปที่ 3.3 ข้อมูลที่ต้องการความถูกต้องและมีความสัมพันธ์กันชัดเจน เช่น ร้านค้า ทีม สินค้า และคำสั่งซื้อ เก็บใน MySQL ส่วนข้อมูลที่เพิ่มขึ้นตลอดเวลาและมีรูปแบบหลากหลาย เช่น บทสนทนาและข้อความ เก็บใน MongoDB ตารางใน MySQL มีหน้าที่ดังตารางที่ 3.1 และคอลเลกชันใน MongoDB มีหน้าที่ดังตารางที่ 3.2

ตารางที่ 3.1 ตารางในฐานข้อมูล MySQL

ชื่อตาราง	หน้าที่
tenants	เก็บข้อมูลร้านค้าที่ใช้งานระบบ และค่าตั้งต้นของร้าน เช่น ภาษา และเวลาที่ถือว่าแชทค้างตอบ
tenant_users	เก็บสมาชิกของร้าน ใช้ยืนยันตัวตนและระบุบทบาทของแต่ละคน
tenant_roles	เก็บบทบาทที่ร้านกำหนดเองและสิทธิ์ของแต่ละบทบาท

ชื่อตาราง	หน้าที่
tenant_user_invitations	เก็บคำเชิญเข้าร่วมทีมที่ส่งทางอีเมลและสถานะการตอบรับ
personal_access_tokens	เก็บโทเคนที่ระบบออกให้แอปพลิเคชันหลังเข้าสู่ระบบ
businesses	เก็บธุรกิจภายใต้ร้านค้า พร้อมชื่อแบรนด์ ข้อมูลรับชำระเงิน และการตั้งค่าการทำงาน
platform_credentials	เก็บช่องทางที่ร้านเชื่อมต่อ และข้อมูลที่ใช้รับส่งข้อความกับแพลตฟอร์ม
business_chatbot_settings	เก็บการตั้งค่าแชทบอทของแต่ละธุรกิจ เช่น โหมด บุคลิก และแบบจำลองภาษาที่ใช้
chatbot_directives	เก็บคำสั่งเพิ่มเติมที่ร้านกำหนดให้แชทบอทปฏิบัติ
customers	เก็บข้อมูลลูกค้าจากทุกช่องทาง เช่น ชื่อ เบอร์โทรศัพท์ และที่อยู่จัดส่ง
products	เก็บแคตตาล็อกสินค้าและราคาของแต่ละธุรกิจ
chat_orders	เก็บคำสั่งซื้อที่เปิดจากบทสนทนา พร้อมยอดรวมและสถานะ
tenant_tags	เก็บแท็กและสีที่ร้านใช้จัดกลุ่มบทสนทนา
saved_replies	เก็บข้อความสำเร็จรูปที่เจ้าหน้าที่เลือกส่งได้ทันที
automation_rules	เก็บกฎตอบกลับอัตโนมัติตามคีย์เวิร์ด
conversation_assignments	เก็บข้อมูลว่าบทสนทนาใดมอบหมายให้สมาชิกคนใด
import_runs	เก็บประวัติและสถานะการนำเข้าแชทย้อนหลัง

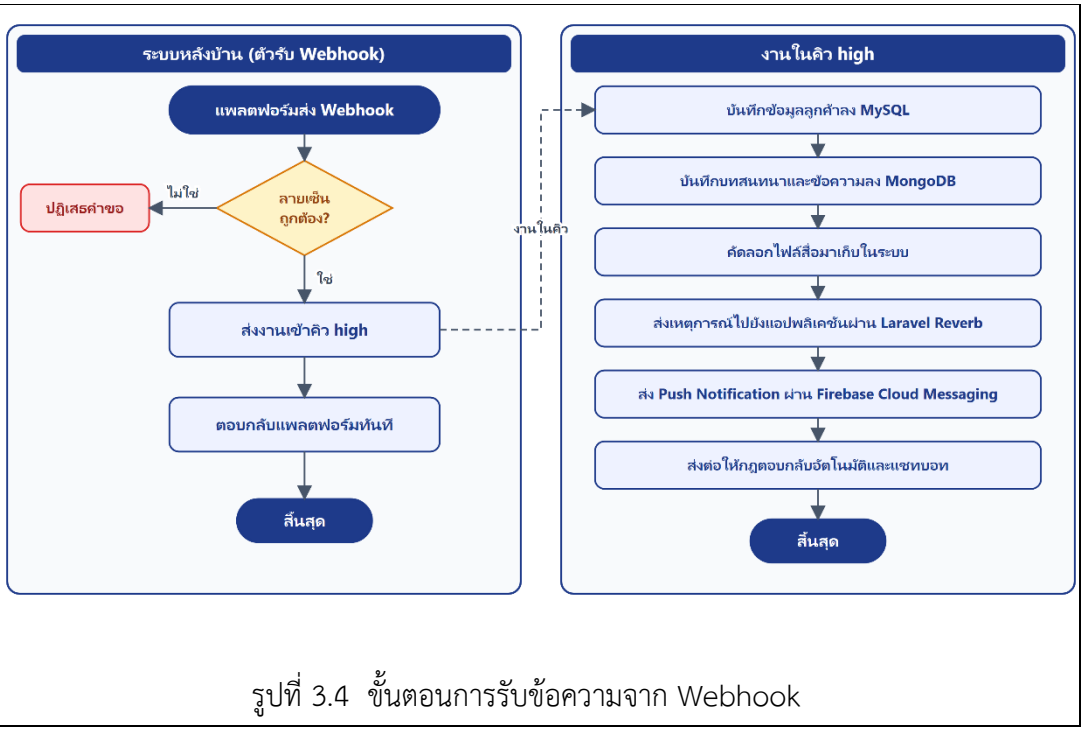
ตารางที่ 3.2 คอลเลกชันในฐานข้อมูล MongoDB

ชื่อคอลเลกชัน	หน้าที่
conversations	เก็บบทสนทนาของลูกค้าแต่ละรายในแต่ละช่องทาง พร้อมสถานะ เช่น ผู้รับผิดชอบ แท็ก จำนวนข้อความที่ยังไม่อ่าน และโหมดแชทบอท
messages	เก็บข้อความขาเข้าและขาออกทุกข้อความ ทั้งข้อความ รูปภาพ วิดีโอ ไฟล์ และสติกเกอร์
knowledge_documents	เก็บเอกสารความรู้ที่ร้านเตรียมไว้ให้แชทบอทใช้ตอบคำถาม
knowledge_chunks	เก็บเอกสารความรู้ที่แบ่งเป็นส่วนย่อยพร้อมเวกเตอร์สำหรับค้นคืน
device_tokens	เก็บโทเคนของอุปกรณ์ที่ใช้ส่ง Push Notification

3.3.2.1 ความสัมพันธ์ระหว่างข้อมูล ร้านค้าหนึ่งแห่งมีสมาชิก บทบาท ธุรกิจลูกค้า และแท็กได้หลายรายการ ธุรกิจหนึ่งแห่งมีช่องทาง สินค้า และกฎตอบกลับได้หลายรายการ และมีการตั้งค่าแชทบอท 1 ชุด ลูกค้าหนึ่งรายมีบทสนทนาและคำสั่งซื้อได้หลายรายการ บทสนทนาหนึ่งรายการมีข้อความได้หลายข้อความ ส่วนเอกสารความรู้หนึ่งฉบับแบ่งเป็นส่วนย่อยได้หลายส่วน

3.3.2.2 การเชื่อมข้อมูลข้ามฐานข้อมูล ข้อมูลใน MongoDB อ้างอิงข้อมูลใน MySQL ด้วยรหัส เช่น บทสนทนาอ้างอิงร้านค้า ธุรกิจ และลูกค้า ส่วนคำสั่งซื้อและผู้รับผิดชอบบทสนทนาใน MySQL อ้างอิงบทสนทนาใน MongoDB ระบบหลังบ้านตรวจความถูกต้องของการอ้างอิงเหล่านี้เอง เพราะฐานข้อมูลทั้งสองไม่บังคับความสัมพันธ์ข้ามกัน

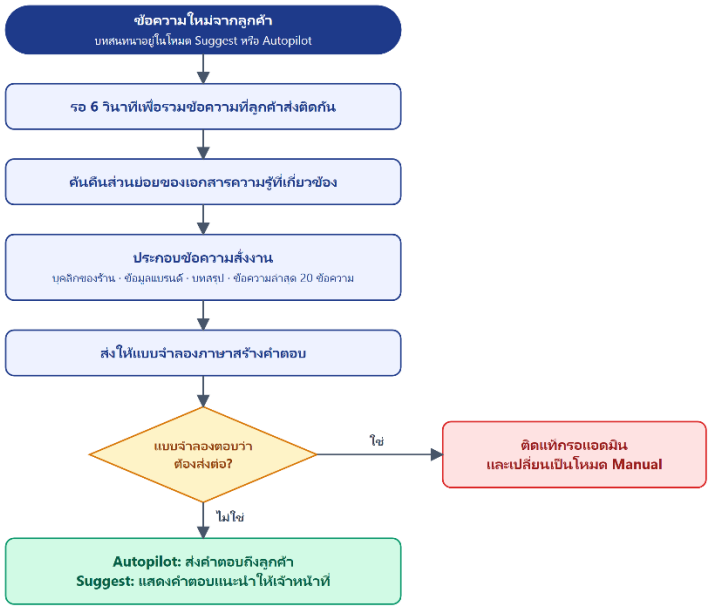
3.3.3 ขั้นตอนการรับข้อความ แสดงดังรูปที่ 3.4



รูปที่ 3.4 ขั้นตอนการรับข้อความจาก Webhook

จากรูปที่ 3.4 แพลตฟอร์มส่ง Webhook มายังระบบหลังบ้าน ระบบตรวจสอบลายเซ็นแล้วส่งงานเข้าคิว high และตอบกลับแพลตฟอร์มทันที จากนั้นงานในคิวจะบันทึกข้อมูลลูกค้าลง MySQL บันทึกบทสนทนาและข้อความลง MongoDB คัดลอกไฟล์สื่อมาเก็บในระบบ ส่งเหตุการณ์ไปยังแอปพลิเคชันผ่าน Laravel Reverb ส่ง Push Notification ไปยังสมาร์ตโฟนผ่าน Firebase Cloud Messaging และส่งข้อความต่อให้กฎตอบกลับอัตโนมัติและแชทบอท

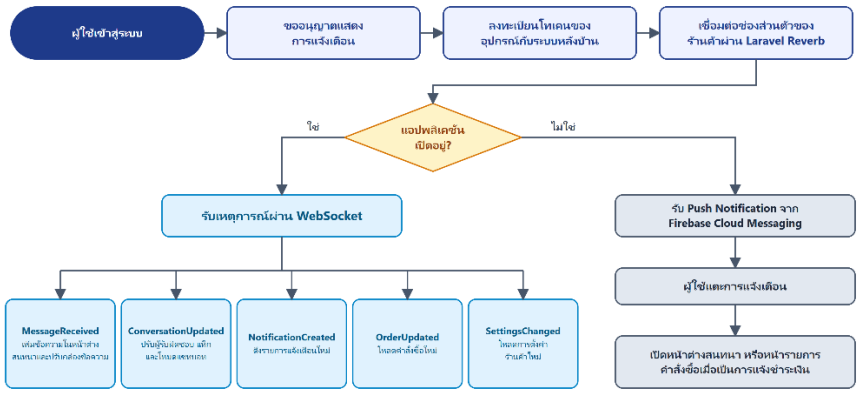
3.3.4 ขั้นตอนการทำงานของแชทบอท แสดงดังรูปที่ 3.5



รูปที่ 3.5 ขั้นตอนการสร้างคำตอบของแชทบอท

จากรูปที่ 3.5 ระบบรอ 6 วินาทีเพื่อรวมข้อความที่ลูกค้าส่งติดกัน แล้วค้นคืนส่วนย่อยของเอกสารความรู้ที่เกี่ยวข้อง ประกอบข้อความสั่งงานจากบุคลิกของร้าน ข้อมูลแบรนด์ บทสรุป และข้อความล่าสุด 20 ข้อความ ส่งให้แบบจำลองภาษาสร้างคำตอบ หากแบบจำลองตอบว่าต้องส่งต่อ ระบบจะติดแท็กรอแอดมินและเปลี่ยนเป็นโหมด Manual

3.3.5 ขั้นตอนการรับข้อมูลแบบเรียลไทม์และการแจ้งเตือน แสดงดังรูปที่ 3.6



รูปที่ 3.6 ขั้นตอนการรับเหตุการณ์และการแจ้งเตือนบนแอปพลิเคชัน

จากรูปที่ 3.6 เมื่อเข้าสู่ระบบ แอปพลิเคชันขออนุญาตแสดงการแจ้งเตือนจากผู้ใช้ลงทะเบียนโทเคนของอุปกรณ์กับระบบหลังบ้าน และเชื่อมต่อช่องส่วนตัวของร้านค้าผ่าน Laravel Reverb เมื่อได้รับเหตุการณ์ แอปพลิเคชันจะทำงาน ดังนี้

3.3.5.1 MessageReceived เพิ่มข้อความในหน้าต่างสนทนาที่เปิดอยู่ และปรับข้อความล่าสุดกับจำนวนที่ยังไม่อ่านในกล่องข้อความ

3.3.5.2 ConversationUpdated ปรับข้อมูลของบทสนทนา เช่น ผู้รับผิดชอบ แท็ก และโหมดแชทบอท

3.3.5.3 NotificationCreated ดึงรายการแจ้งเตือนใหม่

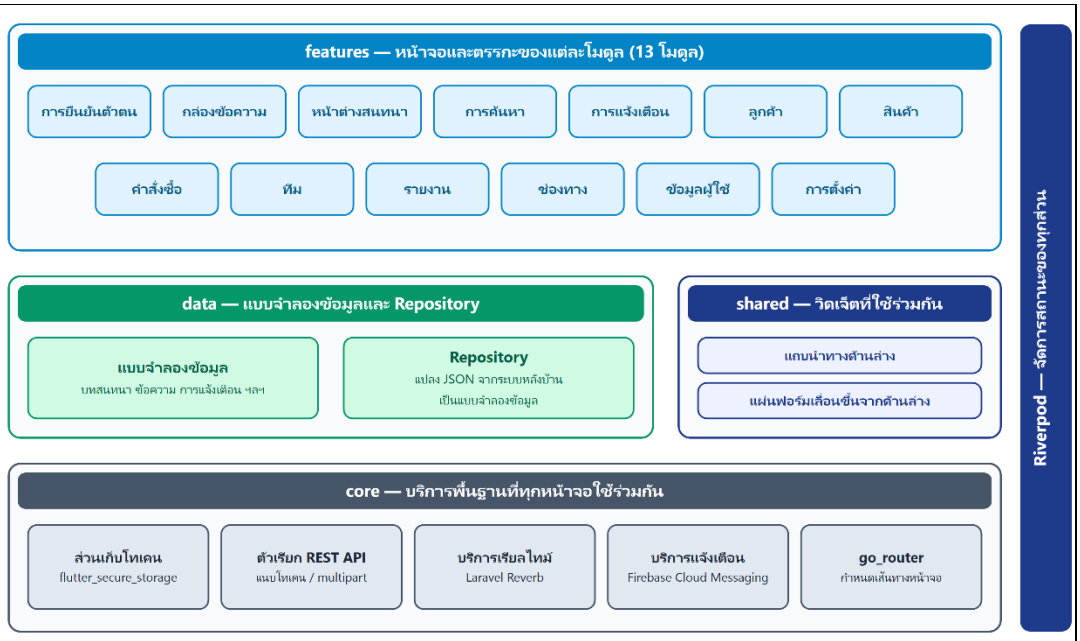
3.3.5.4 OrderUpdated โหลดคำสั่งซื้อของบทสนทนาและรายการคำสั่งซื้อใหม่

3.3.5.5 SettingsChanged โหลดการตั้งค่าของร้านค้าใหม่

3.3.6 Push Notification หากแอปพลิเคชันไม่ได้เปิดอยู่ ผู้ใช้จะได้รับการแจ้งเตือนจาก Firebase Cloud Messaging เมื่อแตะการแจ้งเตือน แอปพลิเคชันจะเปิดหน้าต่างสนทนานั้น หรือเปิดหน้ารายการคำสั่งซื้อของบทสนทนาเมื่อเป็นการแจ้งชำระเงิน ส่วนขณะเปิดแอปพลิเคชันอยู่ จะแสดงแถบแจ้งเตือนด้านล่างพร้อมปุ่มเปิดบทสนทนา และไม่แสดงเมื่อผู้ใช้อยู่ในบทสนทนานั้นแล้ว

3.4 โครงสร้างแอปพลิเคชันบนสมาร์ตโฟน

ผู้จัดทำแบ่งโค้ดของแอปพลิเคชันเป็น 4 ส่วนตามหน้าที่ ดังรูปที่ 3.7



รูปที่ 3.7 โครงสร้างโค้ดของแอปพลิเคชัน

จากรูปที่ 3.7 โค้ดแต่ละส่วนทำหน้าที่ ดังนี้

3.4.1 ส่วน core เป็นบริการพื้นฐานที่ทุกหน้าจอใช้ร่วมกัน ประกอบด้วย ส่วนเก็บโทเคนที่ใช้แพ็กเกจ flutter_secure_storage ตัวเรียก REST API ที่แนบโทเคนให้ทุกคำขอและส่งไฟล์แบบ multipart ได้ บริการเรียลไทม์ บริการแจ้งเตือน และ go_router

3.4.2 ส่วน data เป็นแบบจำลองข้อมูล เช่น บทสนทนา ข้อความ และการแจ้งเตือน และ Repository ที่แปลงข้อมูล JSON จากระบบหลังบ้านเป็นแบบจำลองข้อมูลของแอปพลิเคชัน

3.4.3 ส่วน features เป็นหน้าจอและตรรกะของแต่ละโมดูล 13 โมดูล ได้แก่ การยืนยันตัวตน กล่องข้อความ หน้าต่างสนทนา การค้นหา การแจ้งเตือน ลูกค้า สินค้า คำสั่งซื้อ ทีม รายงาน ช่องทาง ข้อมูลผู้ใช้ และการตั้งค่า

3.4.4 ส่วน shared เป็นวิดเจ็ตที่หลายหน้าจอใช้ร่วมกัน เช่น แถบนำทางด้านล่าง และแผ่นฟอร์มที่เลื่อนขึ้นจากด้านล่างของหน้าจอ

3.4.5 การจัดการสถานะและเส้นทางหน้าจอ การจัดการสถานะใช้ Riverpod โดยสถานะการเข้าสู่ระบบเป็น Provider ต้นทาง เมื่อผู้ใช้ออกจากระบบ Provider ของตัวเรียก API บริการเรียลไทม์ และบริการแจ้งเตือนจะยกเลิกการเชื่อมต่อและล้างข้อมูลของผู้ใช้เดิมทั้งหมด การกำหนดเส้นทางแบ่งเป็นหน้าเข้าสู่ระบบ หน้าสมัครใช้งาน และแถบนำทางด้านล่าง 3 แท็บ ได้แก่ กล่องข้อความ การแจ้งเตือน และข้อมูลผู้ใช้ ซึ่งรวมเมนูไปยังหน้าจอจัดการอื่นทั้งหมด หากได้รับรหัส 401 จากระบบหลังบ้าน แอปพลิเคชันจะลบโทเคนและกลับไปหน้าเข้าสู่ระบบทันที

3.5 โครงสร้างระบบบริการหลังบ้าน

ระบบบริการหลังบ้านพัฒนาด้วย Laravel 10 ประกอบด้วยส่วนต่าง ๆ ดังนี้

3.5.1 ส่วนรับและส่งข้อความ ประกอบด้วย ตัวรับ Webhook ของ LINE และ Meta ที่ตรวจลายเซ็นด้วย HMAC-SHA256 และบริการส่งข้อความออกผ่าน LINE Push API และ Meta Graph API เวอร์ชัน 19.0

3.5.2 ส่วนคิวงานและตัวจัดตารางเวลา ใช้ประมวลผล Webhook แชทบอท และการนำเข้าประวัติแชทในเบื้องหลัง และตรวจหาแชทที่ค้างตอบทุก 1 นาที

3.5.3 ส่วนเรียลไทม์และการแจ้งเตือน ใช้ Laravel Reverb เป็นเซิร์ฟเวอร์ WebSocket และส่งการแจ้งเตือนไปยังโทเคนของอุปกรณ์ผ่าน Firebase Cloud Messaging

3.5.4 ส่วนยืนยันตัวตน ใช้ Laravel Sanctum ออกโทเคนให้แอปพลิเคชันหลังเข้าสู่ระบบ และตรวจสอบโทเคนของ Facebook และ Google ที่แอปพลิเคชันส่งมาก่อนออกโทเคนของระบบ

3.5.5 ส่วนจัดการไฟล์สื่อ คัดลอกไฟล์สื่อขาเข้ามาเก็บในระบบ ย่อรูปภาพด้วย PHP GD และย่อวิดีโอด้วย ffmpeg ก่อนจัดเก็บ

3.5.6 ส่วนแชทบอท ประกอบข้อความสั่งงาน ค้นคืนเอกสารความรู้ และเรียกบริการแบบจำลองภาษา ได้แก่ Ollama, Anthropic Claude, Google Gemini และบริการที่รองรับรูปแบบ OpenAI API

3.5.7 ส่วนการติดตั้งระบบ [รอข้อมูล: เครื่องแม่ข่ายที่ใช้จริง โดเมน และการตั้งค่า Queue worker และ Scheduler บนเครื่องแม่ข่าย]

3.6 การออกแบบ API

ระบบหลังบ้านเปิด REST API ให้แอปพลิเคชันเรียกใช้ประมาณ 99 เส้นทาง แบ่งเป็นหมวดตามหน้าที่ ทุกเส้นทางยกเว้นการสมัครและเข้าสู่ระบบต้องแนบโทเคนที่ได้รับหลังเข้าสู่ระบบ และระบบตรวจสิทธิ์ตามบทบาทของผู้ใช้ก่อนทำงาน ข้อมูลที่รับส่งใช้รูปแบบ JSON หมวดของ API แสดงดังตารางที่ 3.3

ตารางที่ 3.3 หมวดของ API

หมวด	หน้าที่
การยืนยันตัวตน	สมัครใช้งานและสร้างร้านค้าใหม่ เข้าสู่ระบบด้วยอีเมลหรือบัญชี Facebook และ Google ออกจากระบบ และดูข้อมูลผู้ใช้
บทสนทนา	ดูและค้นหาบทสนทนา ดูข้อความ ตอบข้อความและแนบไฟล์ มอบหมาย ติดแท็ก ปักหมุด จัดเก็บ และล็อกบทสนทนา
แชทบอท	เปลี่ยนโหมดแชทบอทของบทสนทนา ขอคำตอบแนะนำ และดูผลวิเคราะห์บทสนทนา
ลูกค้า	ดู ค้นหา และแก้ไขข้อมูลลูกค้า
สินค้า	ดู เพิ่ม แก้ไข และลบสินค้า
คำสั่งซื้อ	เปิดคำสั่งซื้อจากบทสนทนา ดูรายการคำสั่งซื้อ และเปลี่ยนสถานะคำสั่งซื้อ
ทีม	ดูรายชื่อสมาชิก ส่งคำเชิญ เปลี่ยนบทบาท นำสมาชิกออก และโอนความเป็นเจ้าของร้าน
ช่องทาง	เชื่อมต่อ Facebook Page, Instagram, LINE Official Account และ WebChat และนำเข้าแชทย้อนหลัง
การแจ้งเตือนและอุปกรณ์	ดูและอ่านการแจ้งเตือน ลงทะเบียนและยกเลิกโทเคนของอุปกรณ์

หมวด	หน้าที่
รายงาน	ดูตัวชี้วัดการให้บริการและสัดส่วนข้อความของแต่ละช่องทางตามช่วงเวลา
การตั้งค่า	ตั้งค่าร้านค้า ธุรกิจ แท็ก แชทบอท และการแจ้งเตือน

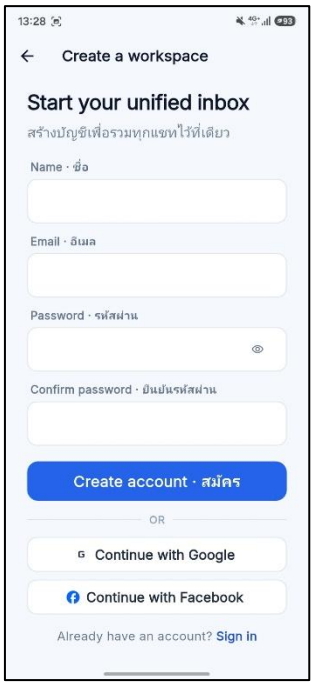
3.7 การพัฒนาแอปพลิเคชันบนสมาร์ตโฟน

ผู้จัดทำพัฒนาแอปพลิเคชันเป็น 13 โมดูลตามโครงสร้างในหัวข้อ 3.4 แต่ละโมดูลทำงานดังนี้

3.7.1 การเข้าสู่ระบบและสมัครใช้งาน ผู้ใช้เข้าสู่ระบบด้วยอีเมลและรหัสผ่าน หรือแตะปุ่ม Facebook หรือ Google ซึ่งเปิดหน้าเข้าสู่ระบบของผู้ให้บริการบนสมาร์ตโฟนโดยตรง แอปพลิเคชันส่งข้อมูลไปตรวจสอบที่ระบบหลังบ้าน เมื่อได้รับโทเคนกลับมาจะเก็บไว้ในพื้นที่จัดเก็บที่เข้ารหัสด้วย flutter_secure_storage แล้วเปิดหน้ากล่องข้อความ ผู้ใช้ใหม่สมัครใช้งานพร้อมสร้างร้านค้าได้ในขั้นตอนเดียว โดยแอปพลิเคชันตรวจรูปแบบอีเมลและกำหนดรหัสผ่านอย่างน้อย 8 ตัวอักษร ดังรูปที่ 3.8 และรูปที่ 3.9

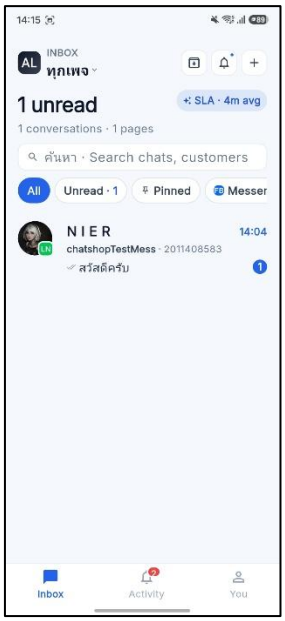


รูปที่ 3.8 หน้าเข้าสู่ระบบ



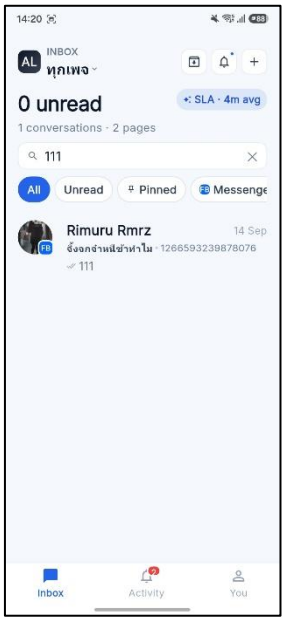
รูปที่ 3.9 หน้าสมัครใช้งาน

3.7.2 กล่องข้อความ แอปพลิเคชันดึงบทสนทนาจากทุกช่องทางมาแสดงในรายการเดียว เรียงบทสนทนาที่ปักหมุดไว้ก่อน แล้วตามด้วยบทสนทนาที่มีข้อความล่าสุด แต่ละรายการแสดงชื่อลูกค้า ช่องทาง ข้อความล่าสุด และจำนวนข้อความที่ยังไม่อ่าน ผู้ใช้กรองรายการได้ตามข้อความที่ยังไม่อ่าน บทสนทนาที่ปักหมุด หรือช่องทาง และเลือกแสดงเฉพาะเพจที่ต้องการได้ ผู้ใช้ปัดรายการเพื่อจัดเก็บหรือลบบทสนทนา และกดค้างเพื่อปักหมุดหรือมอบหมายบทสนทนาให้สมาชิกในทีม เมื่อมีข้อความใหม่ รายการจะปรับตามเหตุการณ์เรียลไทม์ทันทีโดยไม่ต้องรีเฟรช ดังรูปที่ 3.10



รูปที่ 3.10 หน้ากล่องข้อความ

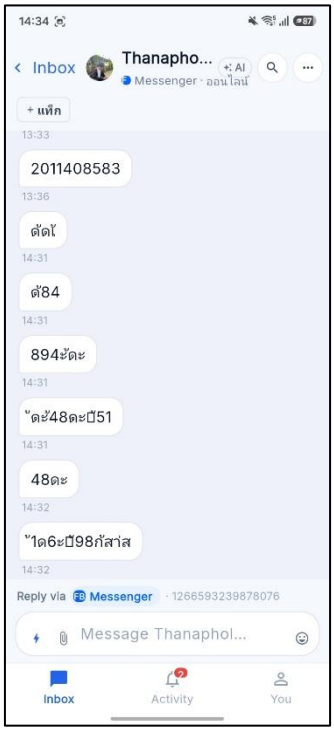
3.7.3 การค้นหา ผู้ใช้พิมพ์คำค้นจากหน้ากล่องข้อความ แอปพลิเคชันส่งคำค้นไปยังระบบหลังบ้านเพื่อค้นหาบทสนทนาของทั้งร้านค้าจากชื่อลูกค้า ข้อความ และแท็ก แล้วแสดงผลลัพธ์ให้แตะเปิดหน้าต่างสนทนาได้ทันที ดังรูปที่ 3.11



รูปที่ 3.11 หน้าค้นหาบทสนทนา

3.7.4 หน้าต่างสนทนา เป็นหน้าที่เจ้าหน้าที่ใช้ตอบลูกค้า แบ่งการทำงานได้ ดังนี้

3.7.4.1 การตอบข้อความ แอปพลิเคชันแสดงข้อความของบทสนทนาและโหลดข้อความเก่าเมื่อเลื่อนขึ้น เจ้าหน้าที่พิมพ์ตอบ แนบรูปภาพ วิดีโอ หรือไฟล์เอกสาร หรือเลือกข้อความสำเร็จรูปมาใช้ได้ทันที แอปพลิเคชันส่งข้อความไปยังระบบหลังบ้าน แล้วระบบหลังบ้านส่งต่อไปยังช่องทางของลูกค้า หากร้านเปิดการล็อกบทสนทนาและบทสนทนานั้นมีผู้อื่นรับผิดชอบอยู่ แอปพลิเคชันจะขอให้ยืนยันก่อนตอบ ดังรูปที่ 3.12



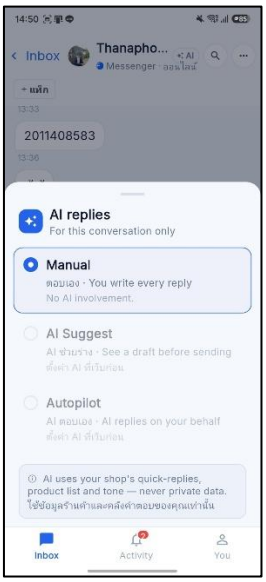
รูปที่ 3.12 หน้าต่างสนทนา

3.7.4.2 การจัดการข้อความ เจ้าหน้าที่กดค้างที่ข้อความเพื่อตอบกลับโดยอ้างอิง ปักหมุด แสดงความรู้สึก หรือใช้ข้อความนั้นเป็นที่อยู่จัดส่งของลูกค้า ข้อความที่ปักหมุดเปิดดูรวมกันได้ ค้นหาข้อความภายในบทสนทนาได้ และเลือกแท็กให้บทสนทนาได้ ดังรูปที่ 3.13



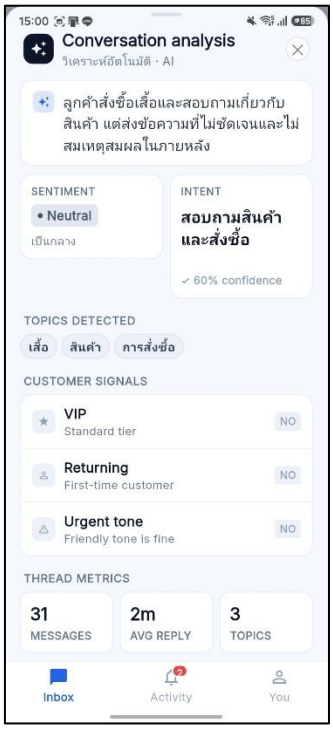
รูปที่ 3.13 เมนูเมื่อกดค้างที่ข้อความ

3.7.4.3 การควบคุมแชทบอท เจ้าหน้าที่เลือกโหมดแชทบอทของบทสนทนาได้ 3 โหมด ได้แก่ Manual, Suggest และ Autopilot แอปพลิเคชันแสดงโหมดใหม่ทันที และย้อนกลับเป็นโหมดเดิมหากระบบหลังบ้านตอบว่าไม่สำเร็จ ในโหมด Suggest แอปพลิเคชันแสดงคำตอบแนะนำเหนือช่องพิมพ์ให้เจ้าหน้าที่เลือกใช้ ส่วนในโหมด Autopilot แอปพลิเคชันซ่อนช่องพิมพ์และแสดงว่าแชทบอทกำลังตอบแทน ดังรูปที่ 3.14



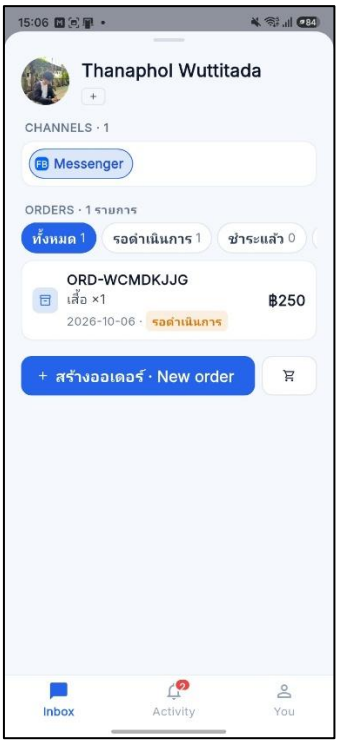
รูปที่ 3.14 การเลือกโหมดแชทบอท

3.7.4.4 การวิเคราะห์บทสนทนา เมื่อเปิดแผ่นวิเคราะห์ แอปพลิเคชันขอให้ระบบหลังบ้านส่งบทสนทนาให้แบบจำลองภาษาวิเคราะห์ แล้วแสดงบทสรุป ความรู้สึก เจตนา หัวข้อที่ลูกค้าสนใจ และสัญญาณสำคัญ เช่น ลูกค้าประจำหรือเรื่องเร่งด่วน ดังรูปที่ 3.15



รูปที่ 3.15 แผ่นวิเคราะห์บทสนทนา

3.7.4.5 ข้อมูลลูกค้าและการเปิดคำสั่งซื้อ แผ่นข้อมูลลูกค้าแสดงช่องทาง ที่อยู่ บันทึกภายใน แท็ก และประวัติคำสั่งซื้อของลูกค้า เจ้าหน้าที่เปิดคำสั่งซื้อจากแผ่นนี้ได้ แบบฟอร์มคำสั่งซื้อช่วยเติมชื่อสินค้าจากแคตตาล็อก คำนวณยอดรวมจากราคาและจำนวนทันที และบันทึกที่อยู่จัดส่ง เมื่อบันทึกแล้ว คำสั่งซื้อจะผูกกับบทสนทนานั้น ดังรูปที่ 3.16 และรูปที่ 3.17

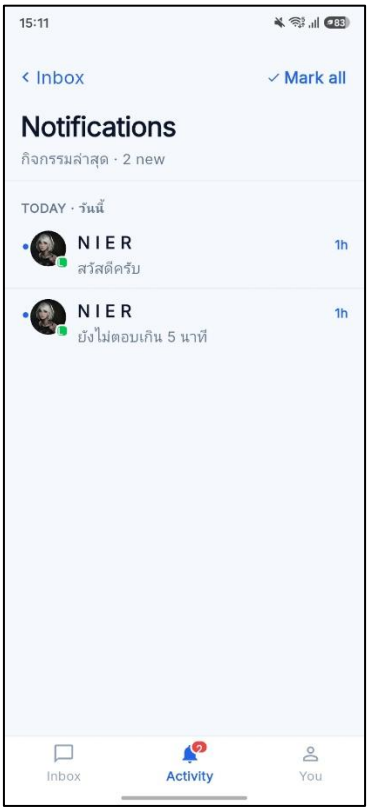


รูปที่ 3.16 แผ่นข้อมูลลูกค้า



รูปที่ 3.17 แบบฟอร์มเปิดคำสั่งซื้อ

3.7.4 การแจ้งเตือน หน้านี้แสดงการแจ้งเตือน 5 ประเภท ได้แก่ เร่งด่วน ข้อความใหม่ ชำระเงินแล้ว การกล่าวถึง และระบบ พร้อมจำนวนที่ยังไม่อ่าน ผู้ใช้แตะรายการเพื่ออ่านและเปิดบทสนทนา หรืออ่านทั้งหมดในครั้งเดียว เมื่อไม่ได้เปิดแอปพลิเคชันอยู่ ผู้ใช้จะได้รับ Push Notification และแตะเพื่อเปิดบทสนทนาได้ตามหัวข้อ 3.3.6 ดังรูปที่ 3.18 และรูปที่ 3.19

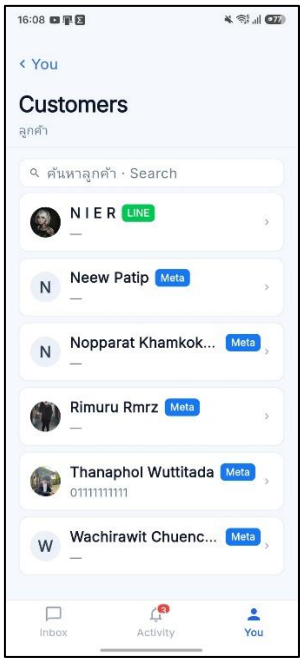


รูปที่ 3.18 หน้าการแจ้งเตือน



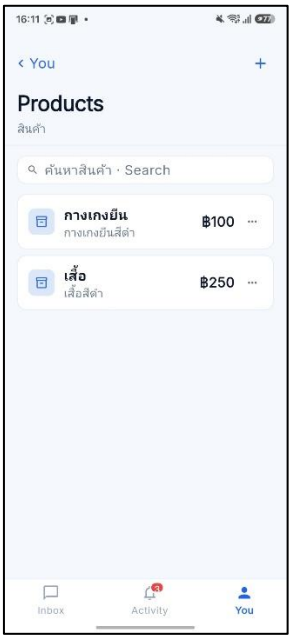
รูปที่ 3.19 Push Notification บนสมาร์ตโฟน

3.7.5 ลูกค้า แสดงรายชื่อลูกค้าจากทุกช่องทาง ค้นหาได้ และแก้ไขเบอร์โทรศัพท์ ที่อยู่ และบันทึกภายในได้ ข้อมูลที่แก้ไขจะแสดงในแผ่นข้อมูลลูกค้าของหน้าต่างสนทนาด้วย ดังรูปที่ 3.20



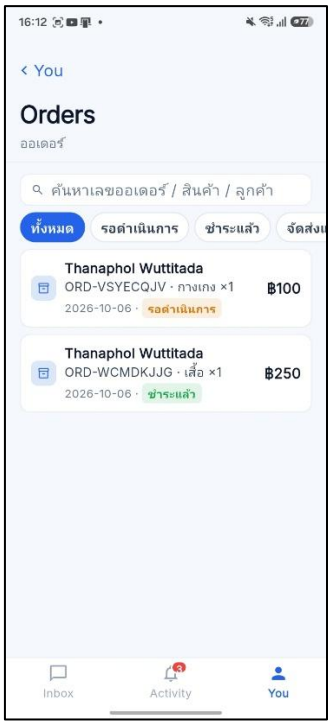
รูปที่ 3.20 หน้าข้อมูลลูกค้า

3.7.6 สินค้า แสดงแคตตาล็อกสินค้าพร้อมราคา ผู้ที่มีสิทธิ์จัดการสินค้าเพิ่ม แก้ไข และลบสินค้าได้ สินค้าในหน้านี้เป็นรายการที่แบบฟอร์มคำสั่งซื้อใช้เติมชื่อและราคาให้ ดังรูปที่ 3.21



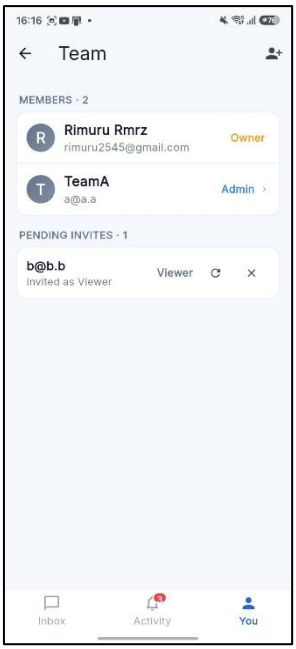
รูปที่ 3.21 หน้าสินค้า

3.7.7 คำสั่งซื้อ แสดงคำสั่งซื้อของร้านแยกแท็บตามสถานะ ได้แก่ รอดำเนินการ ชำระเงินแล้ว จัดส่งแล้ว และยกเลิก ค้นหาได้ และโหลดรายการเพิ่มเมื่อเลื่อนลง เจ้าหน้าที่เปลี่ยนสถานะคำสั่งซื้อได้ และเมื่อสถานะเปลี่ยน แอปพลิเคชันของสมาชิกคนอื่นจะโหลดรายการใหม่ผ่านเหตุการณ์ OrderUpdated ดังรูปที่ 3.22



รูปที่ 3.22 หน้ารายการคำสั่งซื้อ

3.7.8 ทีม เจ้าของร้านเชิญสมาชิกทางอีเมลพร้อมกำหนดบทบาท จัดการคำเชิญที่ค้างอยู่ เปลี่ยนบทบาท นำสมาชิกออก และโอนความเป็นเจ้าของร้านโดยยืนยันด้วยรหัสผ่าน เมนูที่สมาชิกแต่ละคนเห็นจะเป็นไปตามสิทธิ์ของบทบาท ดังรูปที่ 3.23



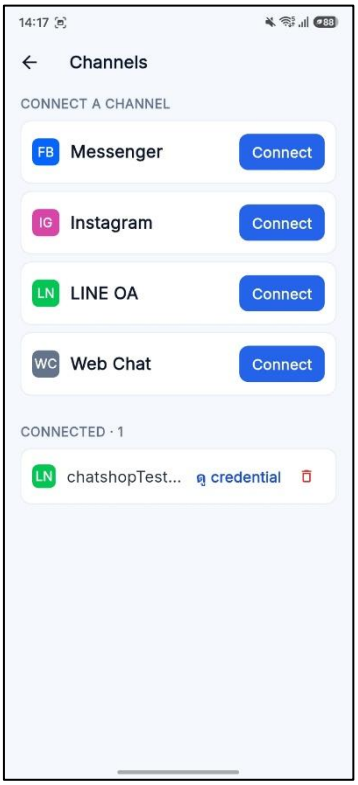
รูปที่ 3.23 หน้าจัดการทีม

3.7.9 รายงาน แสดงตัวชี้วัดการให้บริการ ได้แก่ จำนวนแชททั้งหมด แชทที่ค้างตอบ เวลาเฉลี่ยในการตอบครั้งแรก เวลาเฉลี่ยจนปิดบทสนทนา และสัดส่วนแชทที่ตอบครั้งแรกภายในเวลาที่ร้านกำหนด พร้อมสัดส่วนปริมาณข้อความของแต่ละช่องทาง ผู้ใช้เลือกช่วงเวลาได้ ได้แก่ วันนี้ 7 วัน 30 วัน และรายไตรมาส ดังรูปที่ 3.24



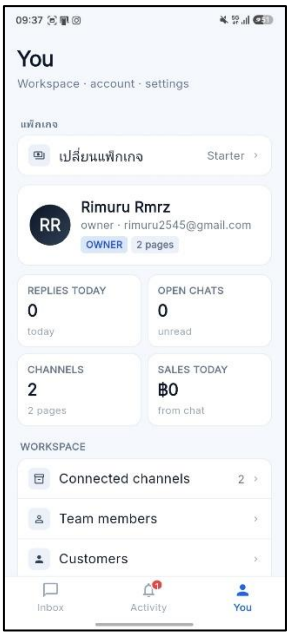
รูปที่ 3.24 หน้ารายงาน

3.7.10 ช่องทาง เชื่อมต่อ Facebook Page และ Instagram ผ่านหน้าอนุญาตสิทธิ์ของ Facebook แล้วเลือกเพจที่ต้องการ เชื่อมต่อ LINE Official Account ด้วย Channel ID และ Channel Secret สร้างโค้ดสำหรับติด WebChat บนเว็บไซต์ และนำเข้าประวัติแชทย้อนหลังได้ไม่เกิน 90 วัน รายละเอียดการเชื่อมต่อแต่ละช่องทางอยู่ในหัวข้อ 3.8 ดังรูปที่ 3.25



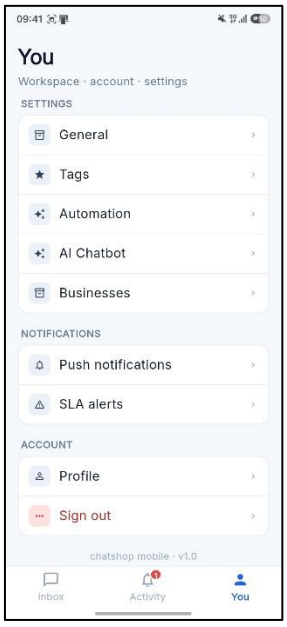
รูปที่ 3.25 หน้าเชื่อมต่อช่องทาง

3.7.11 ข้อมูลผู้ใช้ แสดงสถิติของผู้ใช้ เช่น จำนวนข้อความที่ตอบวันนี้และยอดขายวันนี้ และเป็นเมนูไปยังหน้าจัดการอื่น ได้แก่ ลูกค้า สินค้า คำสั่งซื้อ ทีม รายงาน ช่องทาง และการตั้งค่า รวมถึงออกจากระบบ ดังรูปที่ 3.26



รูปที่ 3.26 หน้าข้อมูลผู้ใช้

3.7.12 การตั้งค่า แบ่งเป็นหน้าย่อยตามเรื่อง เมื่อมีการเปลี่ยนค่า แอปพลิเคชันของสมาชิกคนอื่นจะโหลดค่าใหม่ผ่านเหตุการณ์ SettingsChanged เมนูการตั้งค่าแสดงดังรูปที่ 3.27



รูปที่ 3.27 เมนูการตั้งค่า

3.7.13.1 ร้านค้า แก้ไขข้อมูลทั่วไปของร้าน

3.7.13.2 ธุรกิจ เพิ่มและแก้ไขธุรกิจภายใต้ร้าน และผูกเพจเข้ากับธุรกิจ

3.7.13.3 ระบบอัตโนมัติ เปิดหรือปิดการล็อกบทสนทนา และกำหนดโหมดเริ่มต้นของแชทบอท

3.7.13.4 แชทบอท เลือกผู้ให้บริการและแบบจำลองภาษา กรอกคีย์ API และทดสอบการเชื่อมต่อ ดังรูปที่ 3.28



รูปที่ 3.28 หน้าตั้งค่าแชทบอท

3.7.13.5 แชทค้างตอบ เลือกเวลาแจ้งเตือนแชทที่ค้างตอบ 5 10 15 หรือ 20 นาที

3.7.13.6 การแจ้งเตือน ตั้งค่ารายผู้ใช้ เช่น เสียง Push Notification การแจ้งเฉพาะบทสนทนาที่ได้รับมอบหมาย และปิดการแจ้งเตือนบางช่องทาง

3.7.13.7 แท็ก เพิ่ม แก้ไข และลบแท็กพร้อมสี เมื่อลบแท็ก ระบบจะถอดแท็กนั้นออกจากทุกบทสนทนา

3.8 การตั้งค่าแพลตฟอร์มภายนอก

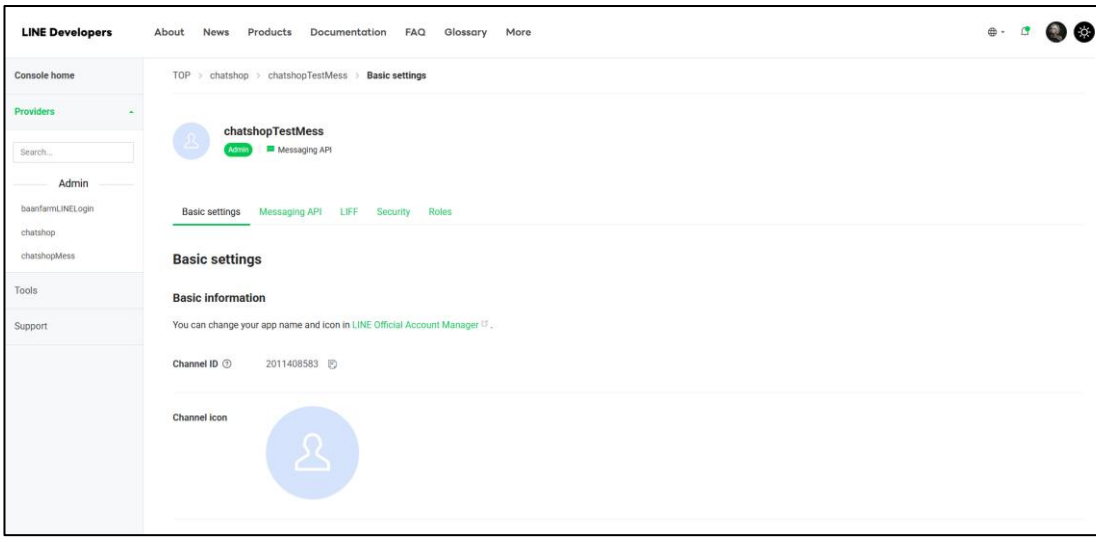
ก่อนใช้งานระบบ ผู้จัดทำต้องสร้างและตั้งค่าบัญชีบนแพลตฟอร์มภายนอก เพื่อให้ระบบหลังบ้านรับส่งข้อความกับ LINE และ Meta ได้ ให้ผู้ใช้เข้าสู่ระบบด้วย Facebook และ Google ได้ และให้ระบบส่ง Push Notification ได้ ทุก URL ที่ลงทะเบียนกับแพลตฟอร์มต้องเป็น HTTPS ที่

เข้าถึงได้จากอินเทอร์เน็ต เส้นทางในหัวข้อนี้ เช่น /webhook/meta หมายถึงเส้นทางต่อจากโดเมนของระบบหลังบ้าน มีรายละเอียด ดังนี้

3.8.1 LINE Official Account ร้านค้าแต่ละร้านสร้างบัญชี LINE Official Account ของตนเอง แล้วนำค่าที่ได้มากรอกในแอปพลิเคชัน ตั้งแต่วันที่ 4 กันยายน 2567 LINE ไม่อนุญาตให้สร้างช่องทาง Messaging API จาก LINE Developers Console โดยตรง ต้องสร้างผ่านบัญชี LINE Official Account ก่อน (LY Corporation, n.d.) มีขั้นตอน ดังนี้

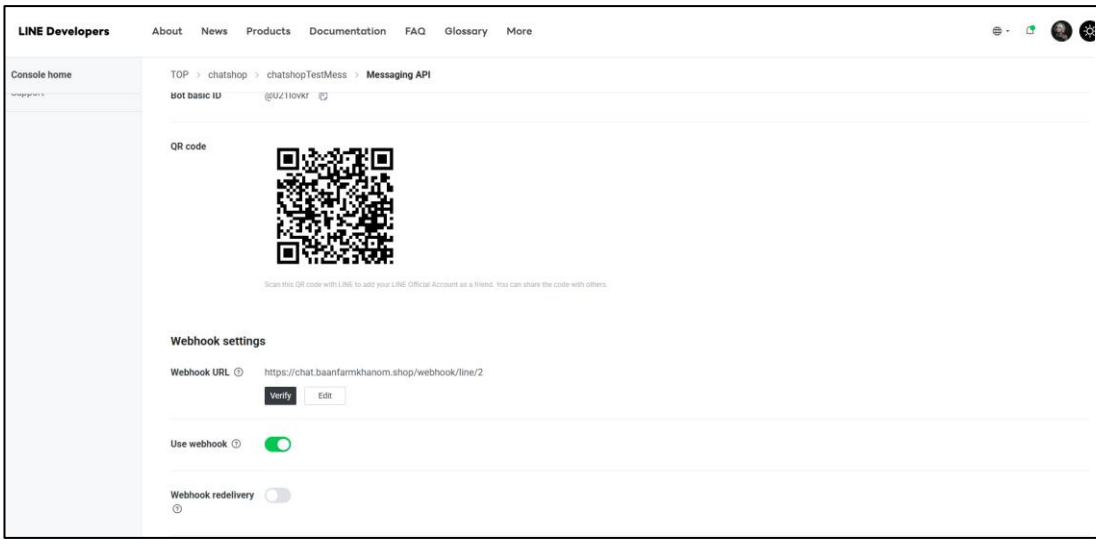
3.8.1.1 สร้างบัญชีและเปิดใช้งาน Messaging API สมัครบัญชี LINE Official Account แล้วเปิดใช้งาน Messaging API ใน LINE Official Account Manager ระบบของ LINE จะสร้างช่องทาง Messaging API ให้อัตโนมัติ

3.8.1.2 คัดลอก Channel ID และ Channel Secret เข้าสู่ LINE Developers Console เลือกช่องทางที่สร้างขึ้น แล้วคัดลอก Channel ID และ Channel Secret จากแท็บ Basic settings ดังรูปที่ 3.29



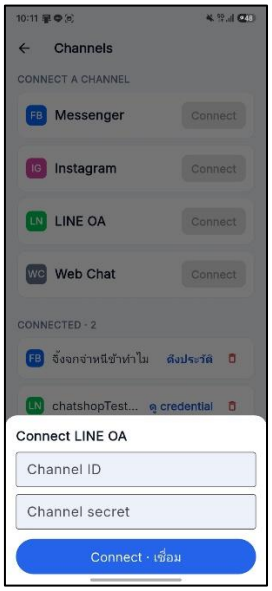
รูปที่ 3.29 Channel ID และ Channel Secret ใน LINE Developers Console

3.8.1.3 ตั้งค่าการตอบกลับ ในแท็บ Messaging API เปิด Use webhook และปิดข้อความตอบกลับอัตโนมัติกับข้อความทักทายของ LINE เพื่อไม่ให้ตอบซ้อนกับแชทบอทของระบบ ดังรูปที่ 3.30



รูปที่ 3.30 การเปิด Use webhook และปิดการตอบกลับอัตโนมัติ

3.8.1.4 เชื่อมต่อในแอปพลิเคชัน ร้านค้ากรอก Channel ID และ Channel Secret ในหน้าเชื่อมต่อช่องทาง ระบบหลังบ้านนำค่าไปขอ Channel Access Token จาก LINE ดึงชื่อและรูปของบัญชี และตั้ง Webhook URL เป็น /webhook/line/{รหัสช่องทาง} ให้อัตโนมัติ ร้านค้าจึงไม่ต้องกรอก Webhook URL เอง จากนั้นระบบใช้ Channel Secret ตรวจลายเซ็นของทุก Webhook ที่ได้รับ ดังรูปที่ 3.31



รูปที่ 3.31 การกรอก Channel ID และ Channel Secret ในแอปพลิเคชัน

3.8.2 Meta ผู้จัดทำสร้างแอปบน Meta for Developers 2 แอปที่แยกหน้าที่กัน ดังตารางที่ 3.4 เหตุที่แยกเป็น 2 แอป เพราะแอปประเภท Business ไม่มี Facebook Login แบบปกติ หากใช้แอปเดียว การเข้าสู่ระบบต้องขอสิทธิ์ business_management ด้วย ทำให้ผู้ใช้ที่ไม่มีเพจหรือ Business Portfolio เข้าสู่ระบบไม่ได้

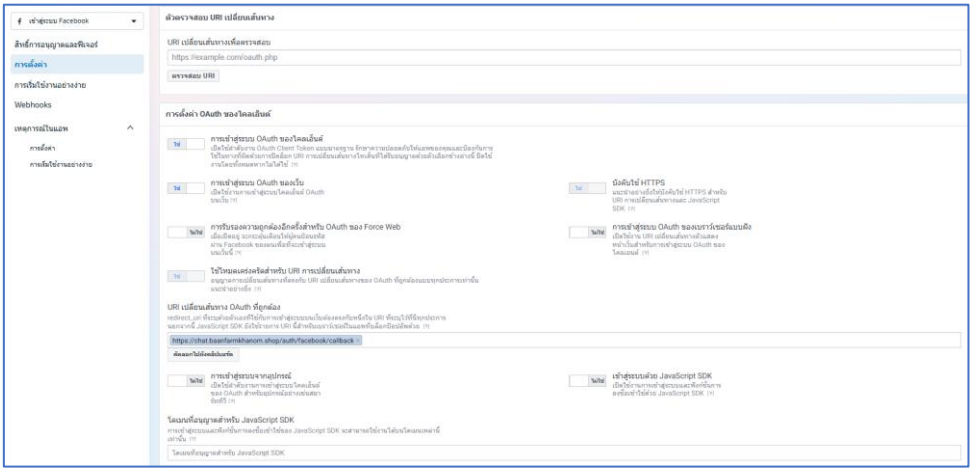
ตารางที่ 3.4 แอปของ Meta ที่ใช้ในระบบ

แอป	ประเภท	หน้าที่
แอปเข้าสู่ระบบ	Consumer	ให้ผู้ใช้เข้าสู่ระบบด้วย Facebook โดยขอสิทธิ์เฉพาะ public_profile และ email
แอปเชื่อมต่อช่องทาง	Business	เชื่อมต่อเพจ Facebook และบัญชี Instagram และรับ Webhook ของข้อความ

3.8.2.1 แอปเข้าสู่ระบบ ตั้งค่าตามขั้นตอน ดังนี้

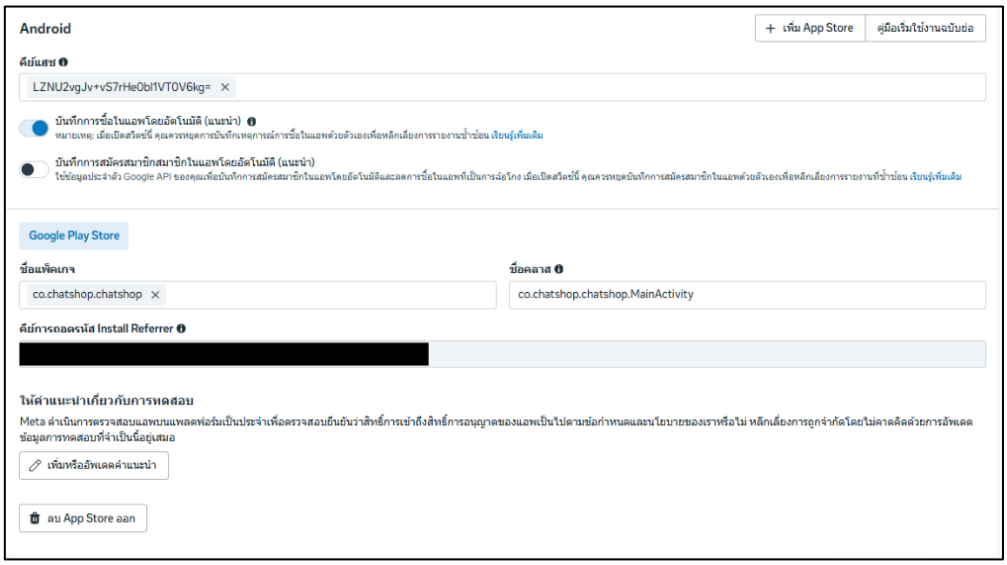
1) สร้างแอปบน Meta for Developers และเพิ่มผลิตภัณฑ์ Facebook Login สำหรับการยืนยันตัวตนผู้ใช้ทั่วไป โดยขอสิทธิ์พื้นฐานเพียง public_profile และ email สำหรับการเข้าสู่ระบบเท่านั้น

2) ตั้งค่า Valid OAuth Redirect URIs ในเมนู Facebook Login เลือก Settings แล้วกำหนด Valid OAuth Redirect URIs เพียง URL เดียว คือ https://<โดเมนระบบหลังบ้าน>/auth/facebook/callback สำหรับการเข้าสู่ระบบผ่านเว็บบราวเซอร์ ดังรูปที่ 3.32



รูปที่ 3.32 การตั้งค่า Facebook Login ของแอปเข้าสู่ระบบ

3) เพิ่มแพลตฟอร์ม Android เพื่อให้แอปพลิเคชันเข้าสู่ระบบได้ โดยในหน้าการตั้งค่าพื้นฐาน (Basic) เพิ่มแพลตฟอร์ม Android ระบุชื่อแพ็กเกจเป็น co.chatshop.chatshop ระบุชื่อคลาสกิจกรรมหลักเป็น co.chatshop.chatshop.MainActivity และกรอก Key Hashes ของใบรับรองลงนาม เพื่อให้แอปพลิเคชันสามารถเข้าสู่ระบบผ่าน Facebook Login SDK ได้โดยตรง ดังรูปที่ 3.33



รูปที่ 3.33 การเพิ่มแพลตฟอร์ม Android ในแอปเข้าสู่ระบบ

4) นำคีย์แอปพลิเคชันไปกำหนดในระบบ คัดลอก App ID และ App Secret ไปกำหนดในไฟล์ .env ของระบบหลังบ้านที่ตัวแปร FACEBOOK_CLIENT_ID และ FACEBOOK_CLIENT_SECRET และคัดลอก App ID กับ Client Token ไปกำหนดในไฟล์ strings.xml ของโปรเจกต์แอปพลิเคชันบนสมาร์ตโฟน

5) การจัดการโหมดการพัฒนา ขณะแอปอยู่ในโหมดพัฒนา (Development Mode) บัญชี Facebook ที่ใช้ทดสอบต้องถูกเพิ่มเป็นผู้ทดสอบ (Testers) ในเมนู App Roles ก่อน หรือสลับเป็นโหมดเผยแพร่ (Live Mode) หลังกรอก URL นโยบายความเป็นส่วนตัว เพื่อให้ผู้ใช้ทั่วไปสามารถเข้าสู่ระบบได้

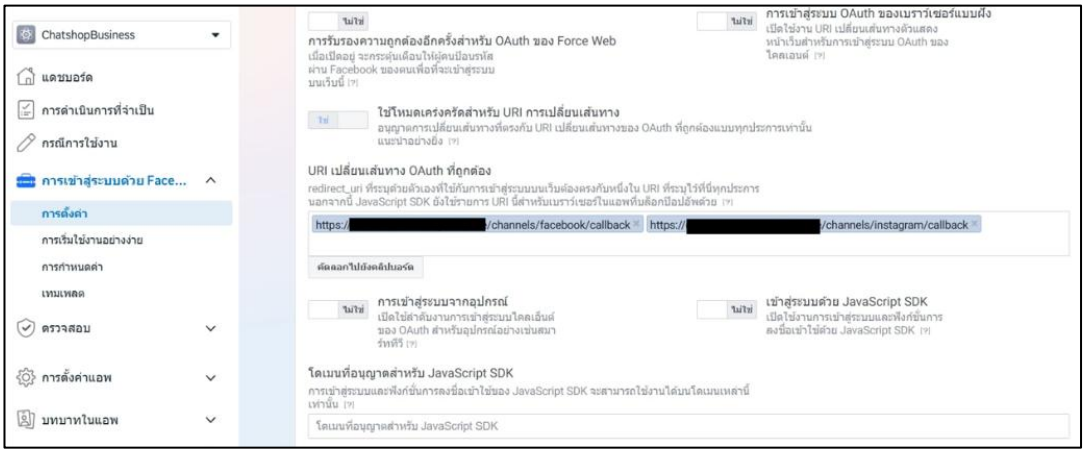
3.8.2.2 แอปเชื่อมต่อช่องทาง ผู้จัดทำสร้างแอปประเภท Business แยกต่างหากอีก 1 แอป สำหรับใช้เชื่อมต่อเพจ Facebook, บัญชี Instagram และรับส่งข้อความผ่าน Messenger โดยเฉพาะ เพื่อไม่ให้ปะปนกับแอปเข้าสู่ระบบทั่วไป มีขั้นตอนการตั้งค่า ดังนี้

1) สร้างแอปประเภท Business แล้วเพิ่มผลิตภัณฑ์ Facebook Login for Business และ Messenger เข้าสู่แอปพลิเคชัน

2) ตั้งค่า Valid OAuth Redirect URIs ในเมนู Facebook Login for Business เลือก Settings แล้วระบุ URL สำหรับเชื่อมต่อช่องทาง 3 เส้นทางตามตารางที่ 3.5 เพื่อรองรับการเชื่อมต่อผ่านระบบเว็บและแอปพลิเคชันบนสมาร์ตโฟน ดังรูปที่ 3.34

ตารางที่ 3.5 Valid OAuth Redirect URIs ของแอปเชื่อมต่อช่องทาง

ใช้สำหรับ	เส้นทาง
เชื่อมต่อเพจ Facebook	/channels/facebook/callback
เชื่อมต่อ Instagram	/channels/instagram/callback
เชื่อมต่อจากแอปพลิเคชัน	/mobile/oauth/facebook/callback



รูปที่ 3.34 การตั้งค่า Valid OAuth Redirect URIs ของแอปเชื่อมต่อช่องทาง

3) สร้าง Configuration ชื่อ connect ในผลิตภัณฑ์ Facebook Login for Business เลือกโทเคนแบบ User access token และขอสิทธิ์ตามตารางที่ 3.6

ตารางที่ 3.6 สิทธิ์ที่ขอใน Configuration connect

สิทธิ์	ใช้ทำอะไร
pages_show_list	ดูรายการเพจที่ผู้ใช้เป็นผู้ดูแล
pages_messaging	รับและส่งข้อความในนามของเพจ
pages_manage_metadata	สมัครรับ Webhook ให้เพจ
pages_read_engagement	อ่านข้อมูลพื้นฐานของเพจ
instagram_basic	อ่านข้อมูลบัญชี Instagram ที่ผูกกับเพจ
instagram_manage_messages	รับและส่งข้อความของบัญชี Instagram
business_management	เข้าถึงเพจที่อยู่ภายใต้ Business Portfolio

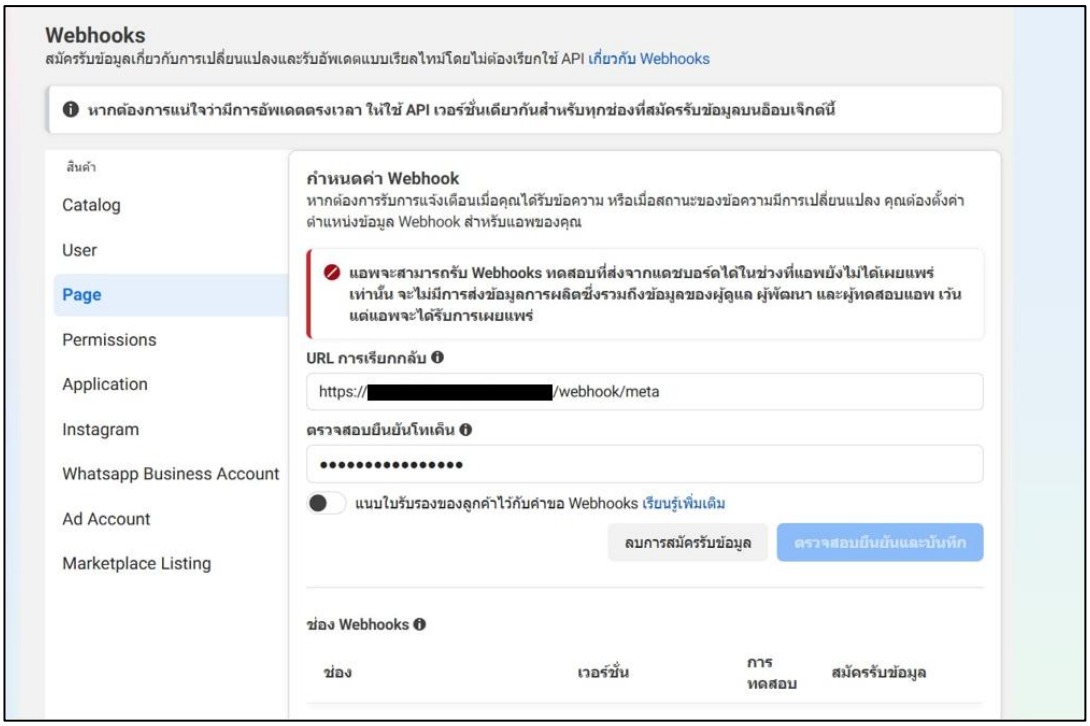
4) คัดลอก App ID, App Secret และ Configuration ID ไปกำหนดในระบบหลังบ้าน

5) ขณะแอปอยู่ในโหมดพัฒนา สิทธิ์เหล่านี้ใช้ได้เฉพาะบัญชีที่มีบทบาทในแอป หากเปิดให้ร้านค้าทั่วไปใช้ ต้องผ่าน App Review และการยืนยันธุรกิจของ Meta

3.8.2.3 Webhook ของ Meta ตั้งค่าที่แอปเชื่อมต่อช่องทาง ในเมนู Messenger แล้วเลือก Webhooks มีขั้นตอน ดังนี้

1) กรอก Callback URL เป็น /webhook/meta

2) กรอก Verify Token เป็นข้อความลับที่ผู้จัดทำกำหนดขึ้นเอง และกำหนดค่าเดียวกันไว้ในระบบหลังบ้าน เมื่อกดยืนยัน Meta จะส่งคำขอ GET มาตรวจค่านี้ ดังรูปที่ 3.35

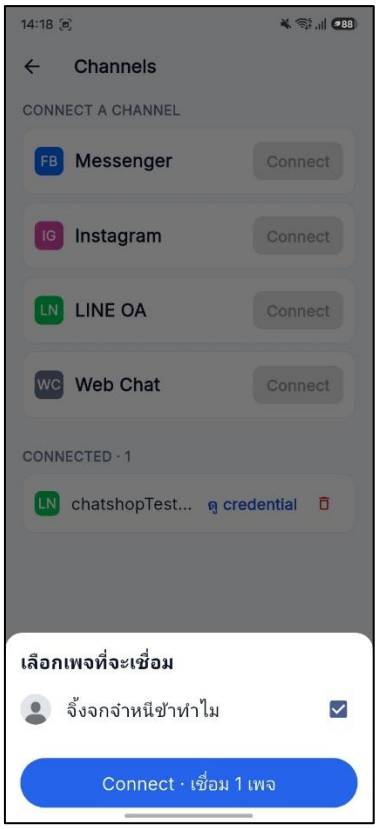


รูปที่ 3.35 การตั้งค่า Webhook ของ Meta

3) สมัครรับฟิลด์ messages และ messaging_postbacks ของเพจ และฟิลด์ messages ของ Instagram

4) ระบบหลังบ้านใช้ App Secret ของแอปนี้ตรวจลายเซ็น X-Hub-Signature-256 ของทุก Webhook หากกำหนด App Secret ของแอปเข้าสู่ระบบแทน ลายเซ็นจะไม่ตรงและระบบจะไม่รับข้อความ

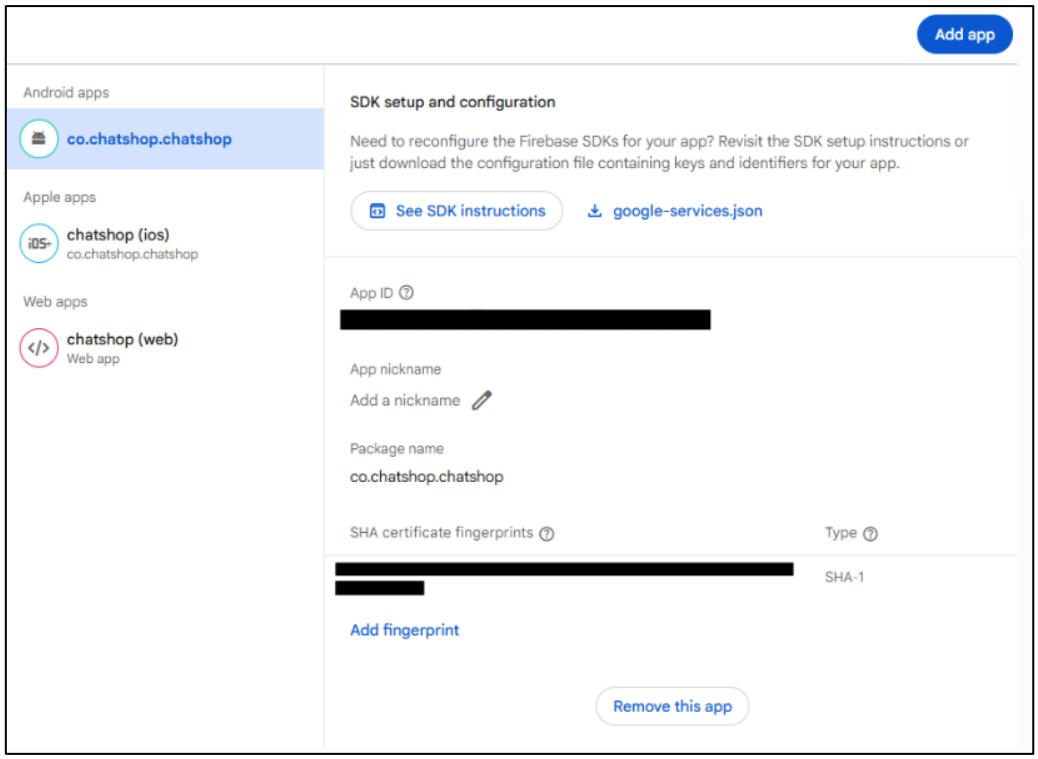
3.8.2.4 เชื่อมต่อในแอปพลิเคชัน ร้านค้าเปิดหน้าเชื่อมต่อช่องทางแล้วแตะปุ่มเชื่อมต่อ Facebook หรือ Instagram แอปพลิเคชันเปิดหน้าอนุญาตสิทธิ์ของ Facebook เมื่ออนุญาตแล้ว ระบบหลังบ้านแลกเป็นโทเคนระยะยาว ดึงรายการเพจที่ผู้ใช้เป็นผู้ดูแลมาให้เลือก แล้วบันทึก Page Access Token และสมัครรับ Webhook ให้เพจนั้นอัตโนมัติ บัญชี Instagram ต้องเป็นบัญชีแบบมืออาชีพที่ผูกกับเพจ Facebook เพราะระบบเชื่อมต่อ Instagram ผ่านเพจที่ผูกไว้ หากโทเคนหมดอายุ ระบบจะปิดช่องทางนั้น ดังรูปที่ 3.36



รูปที่ 3.36 การเลือกเพจที่ต้องการเชื่อมต่อ

3.8.3 Firebase และ Google ผู้จัดทำสร้างโปรเจกต์ Firebase 1 โปรเจกต์ ใช้ทั้งส่ง Push Notification และเข้าสู่ระบบด้วย Google มีขั้นตอน ดังนี้

3.8.3.1 เพิ่มแอปพลิเคชัน Android ด้วยชื่อแพ็กเกจ co.chatshop.chatshop ใส่ SHA-1 ของใบรับรองที่ใช้ลงนามแอปพลิเคชัน แล้วดาวน์โหลดไฟล์ google-services.json ไปไว้ในโปรเจกต์ Flutter ดังรูปที่ 3.37

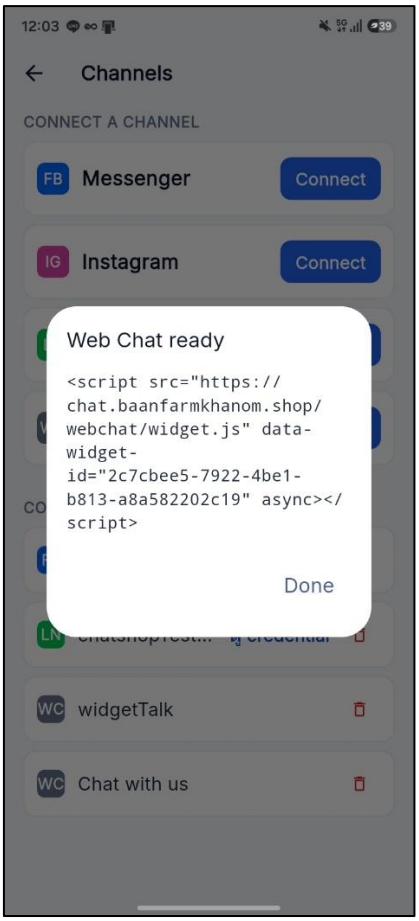


รูปที่ 3.37 โปรเจกต์ Firebase และแอปพลิเคชัน Android

3.8.3.2 สร้างคีย์ของบัญชีบริการจาก Project settings แล้วนำไฟล์คีย์ไปไว้ที่ระบบหลังบ้าน เพื่อให้ระบบหลังบ้านส่งการแจ้งเตือนผ่าน Firebase Cloud Messaging ได้

3.8.3.3 ตั้งค่าหน้าขอความยินยอม OAuth ใน Google Cloud Console ของโปรเจกต์เดียวกัน โดยขอสิทธิ์ email และ profile แอปพลิเคชันใช้ Google Sign-In รับโทเคน แล้วส่งให้ระบบหลังบ้านตรวจสอบกับ Google ก่อนออกโทเคนของระบบ

3.8.4 WebChat ไม่ต้องตั้งค่าบนแพลตฟอร์มภายนอก ร้านค้าสร้างช่องทาง WebChat จากแอปพลิเคชัน กำหนดชื่อ ข้อความทักทาย และสีของหน้าต่างแชท แล้วนำโค้ดสคริปต์ที่ระบบสร้างให้ไปติดบนเว็บไซต์ของร้าน ดังรูปที่ 3.38



รูปที่ 3.38 โค้ดสำหรับติด WebChat บนเว็บไซต์